User Manual

Maker Edition – self-built devices

Battery testing with ZKE EBC testers – stand-alone, recorded on SD, operated from the browser

Firmware	2.0.0 (first stable V2 release, 9 November 2026)
Hardware	Any ESP32-S3 with 16 MB flash; reference design: EBCTester board, schematic revision 1
Firmware mode	MakerMode (GPIO 47 low or open)
Supported testers	ZKE EBC-A20, EBC-A40L, EBC-A50L
Document	Edition November 2026, English
Project	github.com/Gemelon/EBCTester

© 2026 Thomas Stoll. Software and documentation licensed under the Apache License 2.0. EBCTester is an independent hobby project and not a product of, or supported by, ZKETECH.

Contents

Part I – Operating Manual 4

1 Introduction 5
- 1.1 What EBCTester does 5
- 1.2 Main features 5
- 1.3 Factory and Maker devices 6
- 1.4 Supported testers 6
- 1.5 What you need 7
- 1.6 Safety information 7

2 Building and first start 8
- 2.1 Hardware requirements 8
- 2.2 Installing the firmware 8
- 2.3 First start and login 9
- 2.4 Configuring and activating the hardware 10
- 2.5 Connecting to your home Wi-Fi (optional) 12
- 2.6 Selecting the tester model 12
- 2.7 Date and time 12
- 2.8 LEDs 13
- 2.9 The web interface at a glance 13

3 Maker Settings reference 15
- 3.1 EBCTester 16
- 3.2 Network, OTA and web 16
- 3.3 SD card 17
- 3.4 Real-time clock 17
- 3.5 Activation and validation 18

4 Battery tester section 19
- 4.1 Measurements 19
- 4.2 Battery 20
- 4.3 Manual control 22
- 4.4 Automatic control (test plans) 23
- 4.5 Internal resistance (DCIR) 25

5 Administration section 29
- 5.1 Device information 29
- 5.2 Test run management 30
- 5.3 SD card 31
- 5.4 Updates 32
- 5.5 Settings 33
- 5.6 Maker Settings 38
- 5.7 Debug 39

6 Typical workflows 40
- 6.1 Measuring the capacity of a battery 40
- 6.2 Measuring internal resistance at 50 % state of charge 40
- 6.3 Backing up your data 40
- 6.4 Changing the hardware of a Maker device 40

7 Troubleshooting 41

Part II – Technical Reference 43

8 System architecture 44
- 8.1 Firmware structure 44
- 8.2 Data flow 44

9 Start-up and device configuration 45

9.1 Start-up sequence . . . 45
9.2 Firmware modes . . . 45
9.3 GPIO 47 detection . . . 46
9.4 Configuration storage . . . 46
9.5 Firmware defaults . . . 47

10 Hardware . . . 49
10.1 Block diagram . . . 49
10.2 Circuit description . . . 49
10.3 Connectors . . . 51
10.4 Test points . . . 52
10.5 GPIO assignment of the board . . . 52
10.6 Pins and wiring rules . . . 52
10.7 Bill of materials . . . 53
10.8 Printed circuit board . . . 54

11 Building the hardware . . . 58
11.1 Example: the prototype . . . 58
11.2 Replacing board components by modules . . . 59
11.3 Wiring . . . 60
11.4 Commissioning . . . 60

12 Firmware installation and build . . . 62
12.1 Installing a release . . . 62
12.2 Building from source . . . 62
12.3 Build configuration (menuconfig) . . . 62
12.4 Partition table . . . 63
12.5 Status LEDs . . . 63

13 Communication with the tester . . . 64
13.1 Serial interface . . . 64
13.2 Model detection . . . 64
13.3 Operating modes . . . 64

14 Data storage . . . 65
14.1 SQLite database on the SD card . . . 65
14.2 MariaDB backup database . . . 65

15 Interfaces . . . 67
15.1 HTTP API . . . 67
15.2 Home Assistant MQTT topics . . . 68

16 Firmware updates and publishing . . . 69
16.1 Update procedure . . . 69
16.2 Manifest format . . . 69
16.3 Publishing your own builds . . . 69

17 Security and limitations . . . 70
17.1 Security . . . 70
17.2 Known limitations (firmware 2.0.0) . . . 70
17.3 License . . . 70

Appendix A – Language files . . . 71

Appendix B – Glossary . . . 72

Appendix C – Schematic . . . 73

Part I – Operating Manual

This part explains how to bring a self-built EBCTester into operation and use it from day to day: wiring requirements, installing the firmware, configuring the hardware in Maker Settings, activating the configuration, and then running tests and managing the recorded data.

1 Introduction

1.1 What EBCTester does

ZKE (ZKETECH) EBC battery testers are popular, affordable instruments for charging and discharging batteries with constant current, constant power or constant voltage and for measuring capacity. Normally they are controlled by the manufacturer's Windows software, which means a PC has to stay connected for the whole test – often for days.

EBCTester replaces that PC. A small controller with an ESP32-S3 microcontroller is connected to the tester's serial interface. It controls the tester, runs complete test plans on its own and records every measurement in a database on a microSD card. You operate it with any web browser – from a phone, a tablet or a PC – either through your home Wi-Fi or through the device's own Wi-Fi access point. You can close the browser at any time: the test continues and keeps being recorded.

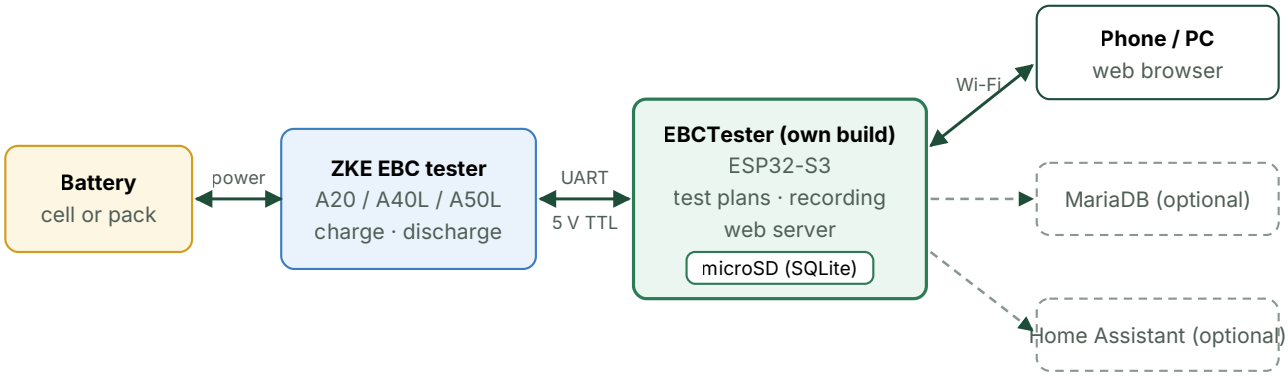


Figure 1: System overview

1.2 Main features

- **No PC needed during a test.** Manual operations and automatic test plans run on the ESP32-S3.
- **Offline recording.** All measurements go into an SQLite database on a FAT32 microSD card. Recording needs a ready SD card, not an Internet connection.
- **Browser interface.** Charts, battery records, test control and administration. Everything, including the chart library, is served by the device itself; no cloud service and no Internet access are required.
- **Test plans** with up to 10 steps: constant-current discharge, constant-power discharge, CC/CV charge, wait and repeat.
- **Internal resistance (DCIR)** by a load-pulse test with three separate results (by default after 1, 5 and 10 seconds), optionally with automatic preparation to a target state of charge.
- **Test-run management:** search, filter, rename and delete recorded runs.
- **Real-time clock:** the DS3231 keeps date and time without Internet access, so measurements receive correct time stamps even when the device runs only on its own access point. Optional on self-built devices. The current device time is shown in the header.
- **Configurable status LEDs:** each WS2812 LED – the two on the board and up to 14 more in an extension chain – can show the tester mode or the message traffic with the tester, and optionally the firmware-update state.
- **Optional** backup to a MariaDB server (for example on a NAS), Home Assistant integration via MQTT and a live debug log.
- **Manual firmware updates** over Wi-Fi: new versions are announced, but only installed after you confirm.

- **Languages:** English and German built in; additional languages can be loaded from the SD card.
- **Maker Settings:** all hardware-related settings (UART, SD and RTC pins, LED, network and update defaults) are configured in the browser and stored on the device – no rebuild of the firmware is needed for your own wiring.

1.3 Factory and Maker devices

The same firmware runs on the original EBCTester board and on self-built devices. At the very first start it decides which of two modes applies, based on the level of GPIO 47:

Mode	Detected when	Behaviour
FactoryMode	GPIO 47 is pulled high (original board: solder jumper JP1 closed to 3.3 V, as delivered)	Starts immediately with the pin assignment of the original board. All functions are available at once.
MakerMode	GPIO 47 is low or not connected	Starts with all hardware outputs switched off. Only the Wi-Fi access point and the web interface run until the maker has entered and activated the hardware configuration in **Maker Settings**.

This manual describes the **Maker Edition**. Owners of an original EBCTester board in its delivered state use the Factory Edition. The operation of the battery-tester functions is identical in both editions; the Maker Edition additionally covers the hardware configuration and contains the detailed technical reference.

The mode is detected only once and then stored. It is detected again only after the device configuration has been reset (see Device reset (section 5.5)).

1.4 Supported testers

The firmware recognises a range of ZKE models by the identifier in their status messages. Only the models marked "enabled" can be controlled; all others are shown as read-only telemetry until their operation has been confirmed.

Model	Max. charge	Max. discharge	Max. power	Status
EBC-A20	18 V / 5 A	30 V / 20 A	85 W	enabled
EBC-A40L	5 V / 40 A	5 V / 40 A	200 W	enabled
EBC-A50L	5 V / 50 A	5 V / 50 A	250 W	enabled
EBC-A05(+), EBC-A10(+/H), EBC-A20+, EBD-M10, EBD-A10H, EBD-M03(+), EBD-M05, EBD-A100(L), EBC-A100L	recognised			read-only

These limits are enforced for every manual command, every plan step and every internal-resistance parameter. The A40L and A50L are low-voltage testers (max. 5 V): use them only with single lithium cells and settings that suit the cell. EBCTester has no battery-chemistry selection – the values you enter must be right for your battery.

> **Note**
> If you own one of the read-only models and can test it with EBCTester, please share your results via the GitHub project so the model can be enabled.

1.5 What you need

- An ESP32-S3 module or board with **16 MB flash** (for example ESP32-S3-WROOM-1(U)-N16R8 or an ESP32-S3-DevKitC-1 N16R8). The partition table does not fit smaller flash sizes.
- A bidirectional **3.3 V ↔ 5 V level shifter** for the tester UART (the tester uses 5 V TTL; ESP32 pins are not 5 V tolerant).
- A microSD socket wired for 4-bit SDMMC or SPI, with 10 kΩ pull-ups on the data and command lines.
- Optionally one or more WS2812 RGB LEDs as status indicators.
- Optionally a DS3231 real-time clock (module or chip) with backup cell, for correct time stamps without Internet.
- A supported ZKE tester, a suitable cable and a 5 V or USB supply for the ESP32.
- A PC with a USB cable for the first installation of the firmware.
- A microSD card formatted as FAT32 (exFAT is not supported). 4–32 GB is ideal.
- A phone, tablet or computer with a current web browser.

1.6 Safety information

> **Warning**
> Batteries store a lot of energy. Wrong settings, damaged cells or poor connections can cause overheating, fire or explosion. Lithium batteries in particular must never be charged above their maximum voltage or discharged below their minimum voltage.

- Enter charge voltages, currents and cut-off values that match the battery's data sheet. EBCTester only checks the limits of the tester, not those of your battery.
- The software does not replace hardware protection (BMS, fuse, temperature monitoring). Long tests are not automatically safe to leave unattended.
- If the serial connection is lost, EBCTester cannot guarantee that the tester stops. Check the tester locally after any connection error.
- Use cables and connectors rated for the test current, and keep the test setup on a non-flammable surface.
- The web interface uses unencrypted HTTP. Use it only in your own trusted network or through the device's protected access point – never expose it to the Internet.
- Wrong pin assignments can damage your hardware, for example when an output is configured on a pin that is driven by another circuit. Check every GPIO in Maker Settings against your wiring before activating the configuration.

2 Building and first start

2.1 Hardware requirements

The firmware does not depend on a particular board. All pins are configured in the browser after the first start. Your hardware must, however, meet the following requirements:

Item	Requirement
Microcontroller	ESP32-S3 with 16 MB flash. Octal PSRAM (N16R8) is optional but recommended. Modules with the "U" suffix need an external 2.4 GHz antenna.
Tester UART	One TX and one RX pin, connected through a 3.3 V ↔ 5 V level shifter (for example TXS0104E) to the tester's TTL port. TX of the ESP32 goes to RX of the tester and vice versa.
Dongle UART	A second TX/RX pair is required by the configuration (reserved for a future PC dongle bridge). It can be left unconnected, but the pins must be free.
SD card	4-bit SDMMC (CLK, CMD, D0–D3) or SPI (MISO, MOSI, CS, CLK); 10 kΩ pull-ups to 3.3 V on CMD and D0–D3 (SDMMC) or MISO/MOSI/CS (SPI). Card detect is optional and must be **high** when a card is inserted.
Status LEDs	Optional WS2812 (GRB) chain of 1–16 LEDs on one output-capable GPIO. The number of LEDs and their functions are set in **Settings → LEDs**.
Real-time clock	Optional DS3231 on I²C (SDA, SCL, optionally INT) with pull-ups to 3.3 V and a backup cell. Disabled by default; enabled in Maker Settings.
GPIO 47	Leave it open or tie it to GND. A high level selects FactoryMode (see Factory and Maker devices (section 1.3)).

The following pins cannot be used for the peripherals: GPIO 26–32 (internal flash), GPIO 33–37 (octal PSRAM), GPIO 19/20 (native USB), GPIO 43/44 (serial console). Avoid the strapping pins 0, 3, 45 and 46. Details are given in Pins and wiring rules (section 10.6). A complete guide to building the circuit yourself – on the original PCB or with breakout modules on a perfboard – is given in Building the hardware (chapter 11).

> **Tip**
> The original EBCTester board is a complete reference design. Its circuit, bill of materials and pin assignment are documented in Part II and can be copied for your own build.

2.2 Installing the firmware

Each GitHub release contains a file `EBCTester-<version>-full-install.bin` with bootloader, partition table and application. Flash it once over USB at address `0x0`, for example with Espressif's esptool:

```
esptool.py --chip esp32s3 --port COM5 write_flash 0x0 EBCTester-2.0.0-full-install.bin
```

If the board is not recognised, start it in download mode: hold BOOT, press and release RESET, release BOOT. Later updates are installed over Wi-Fi (see Updates (section 5.4)). To build the firmware yourself, see Building from source (section 12.2).

> **Note**
> Release images contain no pin assignment that you would have to match: on a Maker device the hardware configuration is entered in the browser. The values offered at the first start are the defaults of the original board.

2.3 First start and login

At the first start the firmware samples GPIO 47 and, with the pin low or open, selects **MakerMode** with an **inactive** configuration. No UART, SD card, LED or other hardware output is driven; only the Wi-Fi access point and the web interface run.

On first start the device opens its own Wi-Fi access point. Its name is `BatteryTester-XXXXXX`, where `XXXXXX` is unique to the device. EBCTester uses two separate passwords:

Password	Used for	Initial value
Access-point password	Connecting to the Wi-Fi network `BatteryTester-XXXXXX`	Set when the firmware was built; the standard firmware uses `ebctester`. If the firmware was built without one, a random password is generated at the first start and shown in the start-up log. Can be changed in Maker Settings.
Login password	Logging in to the web interface (user `admin`) and confirming every settings change	`ebctester`

To read a generated access-point password, connect the device to a PC by USB-C, open a serial terminal (for example the ESP-IDF monitor or PuTTY) and press RESET. The line `Generated setup AP password` shows the password; it stays the same after restarts.

1. Connect your phone or PC to the Wi-Fi network `BatteryTester-XXXXXX` using the access-point password.
2. Open `http://192.168.4.1` in the browser.
3. Log in with user name `admin` and the login password – on a new device `ebctester`. The login page labels this field **Admin password**.
4. Change the login password right away under **Administration → Settings → Login Settings** or in Maker Settings (see Login Settings (section 5.5)).

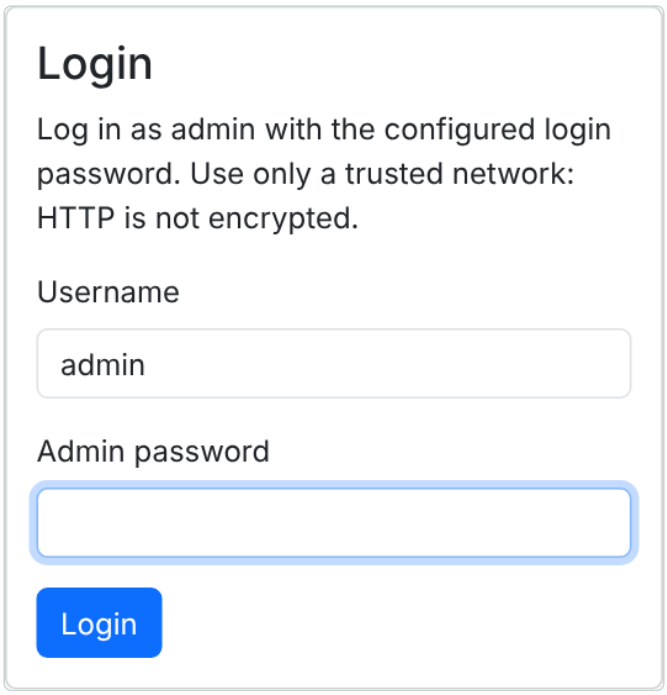


Figure 2: Login page

> **Caution**
> `ebctester` is the same on every device and is published in this manual. Change it on first use. A device reset (see Device reset (section 5.5)) restores this initial password.

A login is valid for 24 hours or 7 days, depending on the setting. It is not extended by activity and ends when the device restarts or you click **Log out**. Running tests and recording are not affected by logging in or out. The login can also be switched off completely (see Login Settings (section 5.5)).

2.4 Configuring and activating the hardware

While the configuration is inactive, the web interface shows only three tabs: **Maker Settings** (opened first), **Settings** and **Device information**, plus **Updates** and – if enabled – **Debug**. The Battery tester section is not available yet.

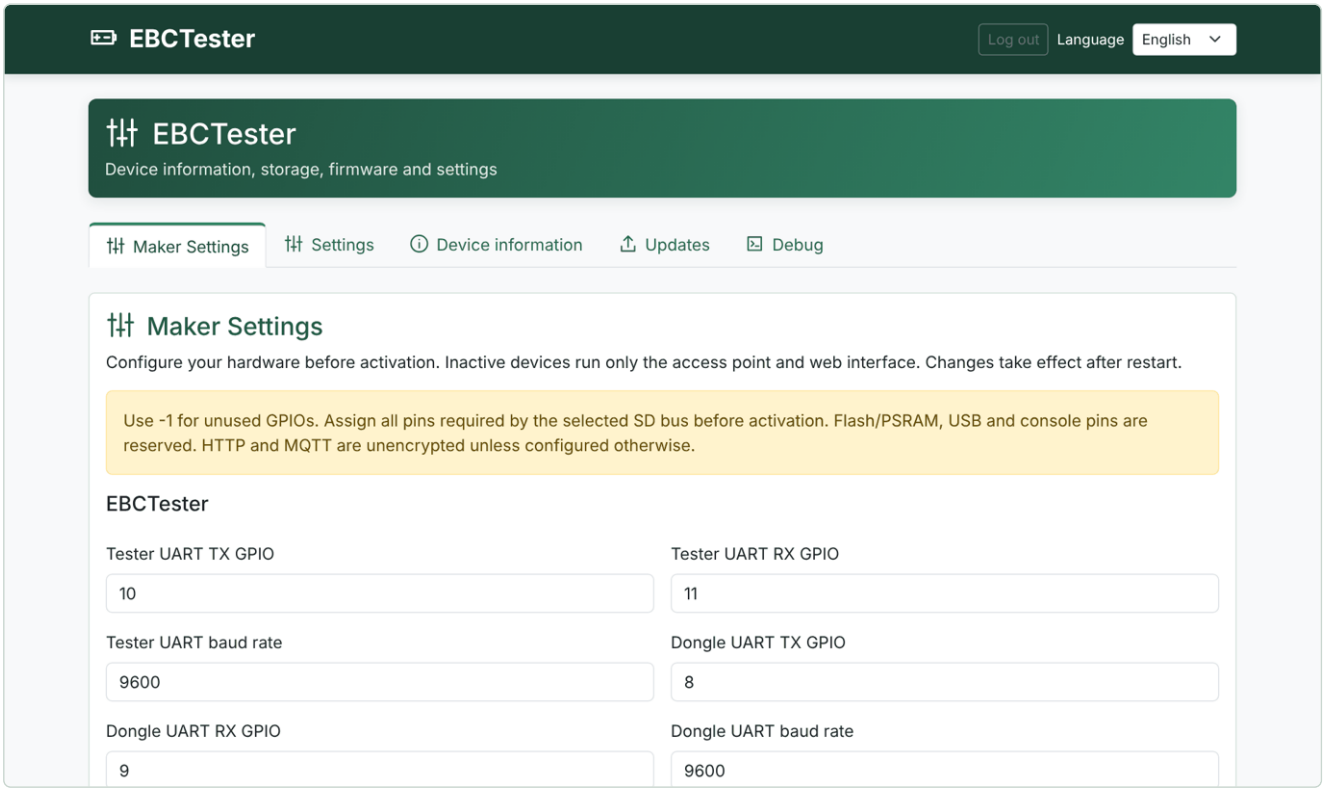


Figure 3: A new Maker device: only the access point and the web interface are running

1. Open **Maker Settings** and enter the GPIO numbers of your wiring: tester UART, dongle UART, SD bus and its pins, card detect, the status LED and – if fitted – the DS3231 real-time clock. Use `-1` for pins that are not connected. All fields are described in Maker Settings reference (chapter 3).
2. Set a personal access-point password and web (login) password. Leave the password fields empty to keep the current passwords.
3. Review the network and update options (time server, firmware manifest URL) and the recording settings.
4. Tick **Configuration active (ConfigAktive)**.
5. Click **Save and restart**, confirm the restart and enter the current login password in the confirmation dialog.
6. The device checks the configuration and restarts. Reconnect to the access point if necessary and log in again. The full interface with the Battery tester section is now available.

When the configuration is saved with ConfigAktive ticked, the firmware checks that every required pin is a valid, output-capable GPIO, that no pin is used twice and that no reserved pin (flash, PSRAM, USB, console) is used. If a check fails, nothing is saved and the message *Invalid configuration. Check passwords, GPIO assignments, reserved pins and selected SD bus.* is shown.

Tip

You can save the configuration without ticking ConfigAktive. The device then restarts and stays inactive – useful to store passwords and Wi-Fi credentials before the wiring is finished.

Caution

Saving the hardware configuration is blocked while a test, plan or recording is running or a firmware update is in progress. Stop the test and wait until the recording has finished first.

2.5 Connecting to your home Wi-Fi (optional)

EBCTester works completely without home Wi-Fi. Connecting it to your network is still useful: you can reach it from every device in the house, the clock is synchronised from the Internet (needed for update checks and to set the real-time clock) and optional services such as MariaDB and MQTT become possible.

1. Go to **Administration → Settings → Configure Wi-Fi**.
2. Enter the network name (SSID) and password and click **Save**. Confirm with the login password when asked.
3. The device connects; its new IP address is shown under **Device information → Wi-Fi IP**. Your router also lists it.

The protected access point remains active as a fallback, so you can always reach the device at `http://192.168.4.1`, even when the home network is out of range. Changing the Wi-Fi channel may briefly interrupt the access-point connection.

> **Note**
> While the configuration is inactive, the device runs the access point only. Wi-Fi credentials can already be saved; the device connects to your network after the configuration has been activated.

2.6 Selecting the tester model

Before the first connection you must tell EBCTester which tester you use – there is no default. Go to **Administration → Settings → EBC tester**, select the model and click **Save tester model**.

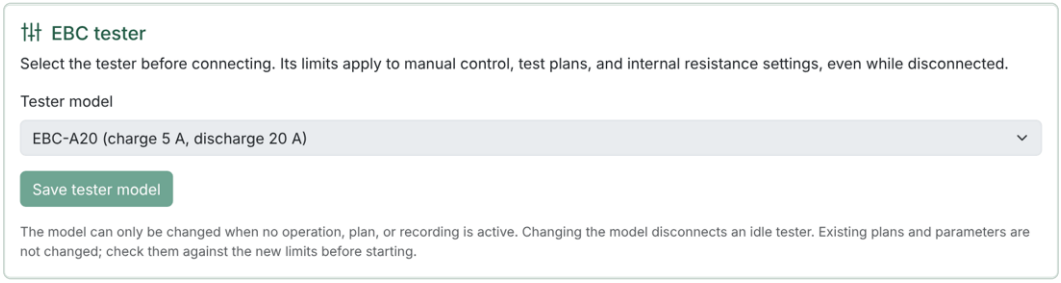


Figure 4: Selecting the tester model

The selected model and its limits apply immediately, even without a connected tester, so you can prepare plans and parameters in advance. The model can only be changed while no operation, plan or recording is active. If the connected tester reports a different supported model, EBCTester shows an error, disconnects and does not reconnect automatically until you correct the selection or click **Connect** again.

2.7 Date and time

Measurements are stored with UTC time stamps. EBCTester takes the time from two sources:

- **Internet time (NTP)** whenever the device is connected to a Wi-Fi network with Internet access. NTP has priority and is checked again every six hours.
- **Real-time clock (DS3231)**. It is read at every start, so the time is correct immediately – also when the device runs only on its own access point. Every successful NTP synchronisation sets the real-time clock again.

The DS3231 is optional on Maker devices and disabled by default. Enable it in Maker Settings (see Real-time clock (section 3.4)). Without it, the time is available only while NTP can be reached.

The header shows the device time with the date below it, converted to the time zone and summer-time rules of your browser. Dashes (`--:--:--`) mean that the device has no valid time yet. Without a valid time, measurements are stored with the device uptime instead of a date.

2.8 LEDs

If WS2812 status LEDs are configured and enabled, each LED shows one function group. The assignment is made in **Settings → LEDs** (see LED settings (section 5.5)). By default LED 1 shows the tester mode and LED 2 the message frames. While the configuration is inactive, all LEDs stay off.

Function group	Display	Meaning
Tester mode	off	Tester not connected
	green	Tester connected and idle
	blue	Manual operation running
	orange	Automatic test plan running
Message frame status	off	No message for more than 5 seconds
	dark green	Messages are being received
	short bright-green flash	One flash for every valid message
	orange	Communication error
Off	off	LED not used

During a firmware update, every LED with the option **OTA override** (as delivered: LED 1) shows the update state instead of its normal function:

Display	Meaning
blue, pulsing	Firmware update is being downloaded
yellow, pulsing	Downloaded firmware is being verified
green, steady	Update verified, device is about to restart
red, blinking	Firmware update failed

The header of the web interface also shows the communication with the tester (see The web interface at a glance (section 2.9)).

2.9 The web interface at a glance

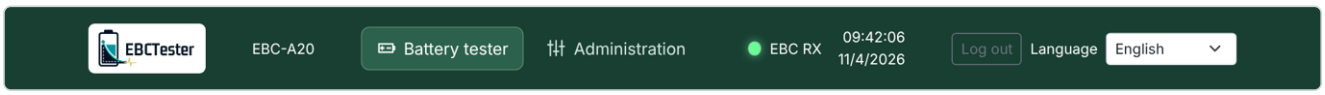


Figure 5: Header bar

The header bar is always visible. From left to right it contains:

Logo	Returns to the start page.
Model	The selected tester model (for example EBC-A20). It turns into a warning if the connected tester does not match or is unknown.

Battery tester / Administration	Switches between the two sections of the interface. Each section remembers its last open tab.
RX indicator	Green while the tester is connected, with a short flash for every valid message received from the tester. Grey = tester not connected, amber = the browser cannot reach the device.
Clock	Device time and date in the time zone of your browser (see Date and time (section 2.7)). Dashes mean that no valid time is available.
Log out	Ends the browser session. Hidden when browser login is switched off.
Language	English, Deutsch and any languages loaded from the SD card. The choice is stored in the browser; switching reloads the page (unsaved form entries are lost, a running test is not affected).

Section	Tabs
Battery tester	Measurements · Battery · Manual · Automatic · Internal resistance
Administration	Device information · Test run management · SD card · Updates · Settings · Debug (only when enabled) · Maker Settings

Numeric input fields are checked as soon as you leave them. A red message below the field explains the allowed range; start and save buttons stay disabled until all fields are valid. When a start button is disabled for another reason (for example tester not connected), a yellow note below it tells you why.

> **Note**
> The screenshots in this manual were created with simulated example data: an EBC-A20 testing a 50 Ah LiFePO4 cell. They show example settings, not recommendations for your battery.

3 Maker Settings reference

The **Maker Settings** tab exists only in MakerMode. On an inactive device it is the first tab; after activation it is found at the end of the Administration section. All values are stored on the device (see Configuration storage (section 9.4)) and take effect after the restart that follows **Save and restart**.

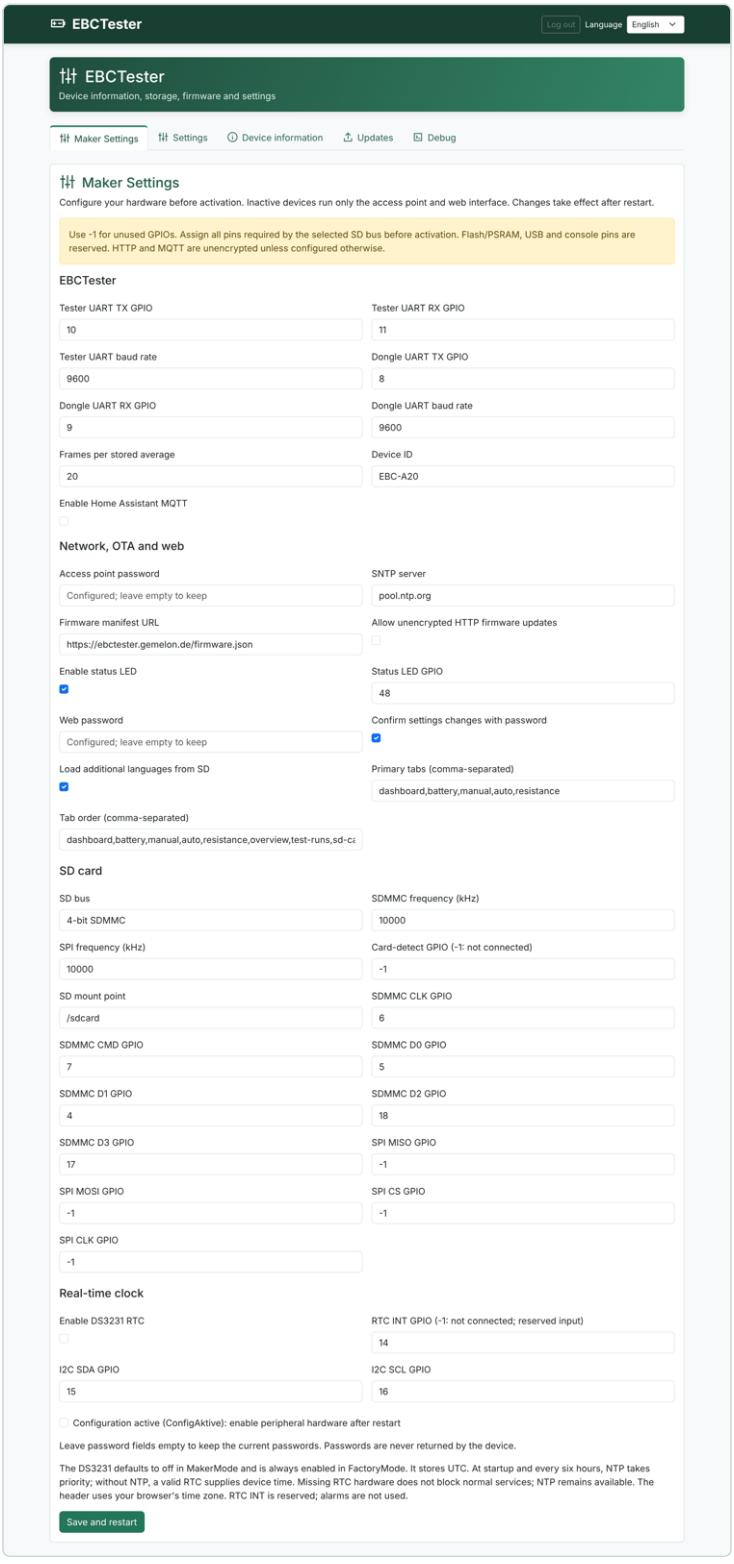


Figure 6: Maker Settings with the pin assignment of the original board

3.1 EBCTester

Field	Range	Meaning
Tester UART TX / RX GPIO	−1 … 48	Pins of UART1 to the tester (ESP32 point of view: TX sends to the tester). Required when active.
Tester UART baud rate	1200 … 115200	The EBC testers use 9600 Bd (8 data bits, odd parity, 1 stop bit). Change only for special adapters.
Dongle UART TX / RX GPIO, baud rate	−1 … 48 / 1200 … 115200	UART2, reserved for the bridge to the manufacturer's PC software (not implemented yet). Two free pins are required when active.
Frames per stored average	1 … 120	The tester sends about one status message per second. This many messages are averaged into one stored measurement (default 20, about one measurement every 20 s). A mode, step, cycle or set-point change always starts a new average.
Device ID	text	Identifier stored with plans and test runs (default `EBC-A20`).
Enable Home Assistant MQTT	on/off	Initial state of the MQTT client. The broker itself is configured under **Settings → Home Assistant MQTT**.

3.2 Network, OTA and web

Field	Range	Meaning
Access point password	8 … 63 characters	Password of the setup access point `BatteryTester-XXXXXX`. Leave empty to keep the current password.
SNTP server	host name	Time server (default `pool.ntp.org`). The time is needed for date stamps and HTTPS update checks.
Firmware manifest URL	HTTPS URL	Address of `firmware.json` that announces new firmware. Empty disables update checks. `http://` is accepted only when unencrypted updates are allowed.
Allow unencrypted HTTP firmware updates	on/off	Only for a trusted test network. Default off.
Enable status LED / Status LED GPIO	on/off, −1 … 48	Data pin of the WS2812 LED chain. The number of LEDs and their functions are set in **Settings → LEDs**.
Web password	8 … 63 printable ASCII	Login password for user `admin`; the same password as in Login Settings. No spaces at the beginning or end. Leave empty to keep it.
Confirm settings changes with password	on/off	When on (default), every settings change asks for the login password, also with browser login switched off.
Load additional languages from SD	on/off	Allows UI translations from the `/languages` folder of the SD card.
Primary tabs	tab ids	Tabs shown in the Battery tester section; all others appear under Administration. Default `dashboard,battery,manual,auto,resistance`.

Field	Range	Meaning
Tab order	tab ids	Display order; the first tab opens when the page loads. Unlisted tabs are appended. Tab ids: `dashboard`, `battery`, `manual`, `auto`, `resistance`, `overview`, `test-runs`, `sd-card`, `updates`, `settings`, `debug`, `maker-settings`.

3.3 SD card

Field	Range	Meaning
SD bus	4-bit SDMMC / SPI	Bus mode of the card socket. Only the pins of the selected bus are used.
SDMMC / SPI frequency	400 … 20000 kHz	Bus clock, default 10 000 kHz. Reduce it for long wires.
Card-detect GPIO	−1 … 48	Input that is **high** when a card is inserted. `−1` = no card-detect switch; the card is then assumed to be present. For a switch that pulls the signal low with a card inserted (as on the original board), enter `−1`.
SD mount point	`/name`	Mount point of the file system, default `/sdcard`. One level only.
SDMMC CLK, CMD, D0 … D3 GPIO	−1 … 48	Pins for 4-bit SDMMC; all six are required with this bus.
SPI MISO, MOSI, CS, CLK GPIO	−1 … 48	Pins for SPI; all four are required with this bus.

3.4 Real-time clock

Field	Default	Meaning
Enable DS3231 RTC	off	Uses a DS3231 as offline time source. Takes effect after the restart and only with an active configuration. (FactoryMode always enables it with the pins of the original board.)
RTC INT GPIO	14	INT/SQW output of the DS3231. Reserved as input only; alarms and wake-up are not used yet. `−1` if not connected.
I2C SDA GPIO	15	I²C data line; required when the RTC is enabled.
I2C SCL GPIO	16	I²C clock line; required when the RTC is enabled. The bus runs at 100 kHz and needs pull-ups to 3.3 V (on most DS3231 modules already fitted).

At every start a valid RTC time is copied to the device clock before recording and the network services start. When NTP is reachable, it has priority: every successful synchronisation (at the latest every six hours) also writes the time to the RTC. Without NTP, the device clock is refreshed from the RTC every six hours. Both clocks run in UTC; the browser converts the time for display.

An RTC that reports an invalid date or a stopped oscillator (for example after the backup cell was empty) is ignored until the next NTP synchronisation sets it again. A missing or faulty RTC is only reported in the log and never blocks the other functions. Without NTP and without a valid RTC, measurements are stored with the device uptime instead of a date.

3.5 Activation and validation

The checkbox **Configuration active (ConfigAktive)** decides whether the hardware is used after the restart. The following checks apply to every save; with ConfigAktive ticked the additional pin checks apply:

Always	Additionally when active
Passwords: AP password empty or 8–63 characters; web password 8–63 printable ASCII characters without leading/trailing spaces	Tester and dongle UART pins, the pins of the selected SD bus and – if enabled – the LED and RTC SDA/SCL pins are set
Every GPIO is −1 or a valid ESP32-S3 GPIO	Output pins are output-capable; RX pins, card detect and optional RTC INT may be input-only
Ranges of baud rates, averaging length, SD frequencies; device ID and SNTP server not empty; mount point of the form `/name`; tab lists contain only `a–z`, `0–9`, `-` and `,`	No pin is assigned twice
Manifest URL empty, `https://` (or `http://` with HTTP allowed), without user name or fragment	No reserved pin: GPIO 19/20 (USB), 26–37 (flash/PSRAM), 43/44 (console)

If a stored configuration that is marked active fails the pin checks at a later start (for example after a firmware change), the device starts inactive and logs *Unsafe peripheral configuration; starting inactive*. Correct the settings in Maker Settings and activate them again.

To return a device to the inactive setup state, untick ConfigAktive and save. The settings are kept.

4 Battery tester section

4.1 Measurements

The **Measurements** tab is the dashboard for recorded test runs. All data comes from the database on the SD card, so the page also works on the access point without Internet.

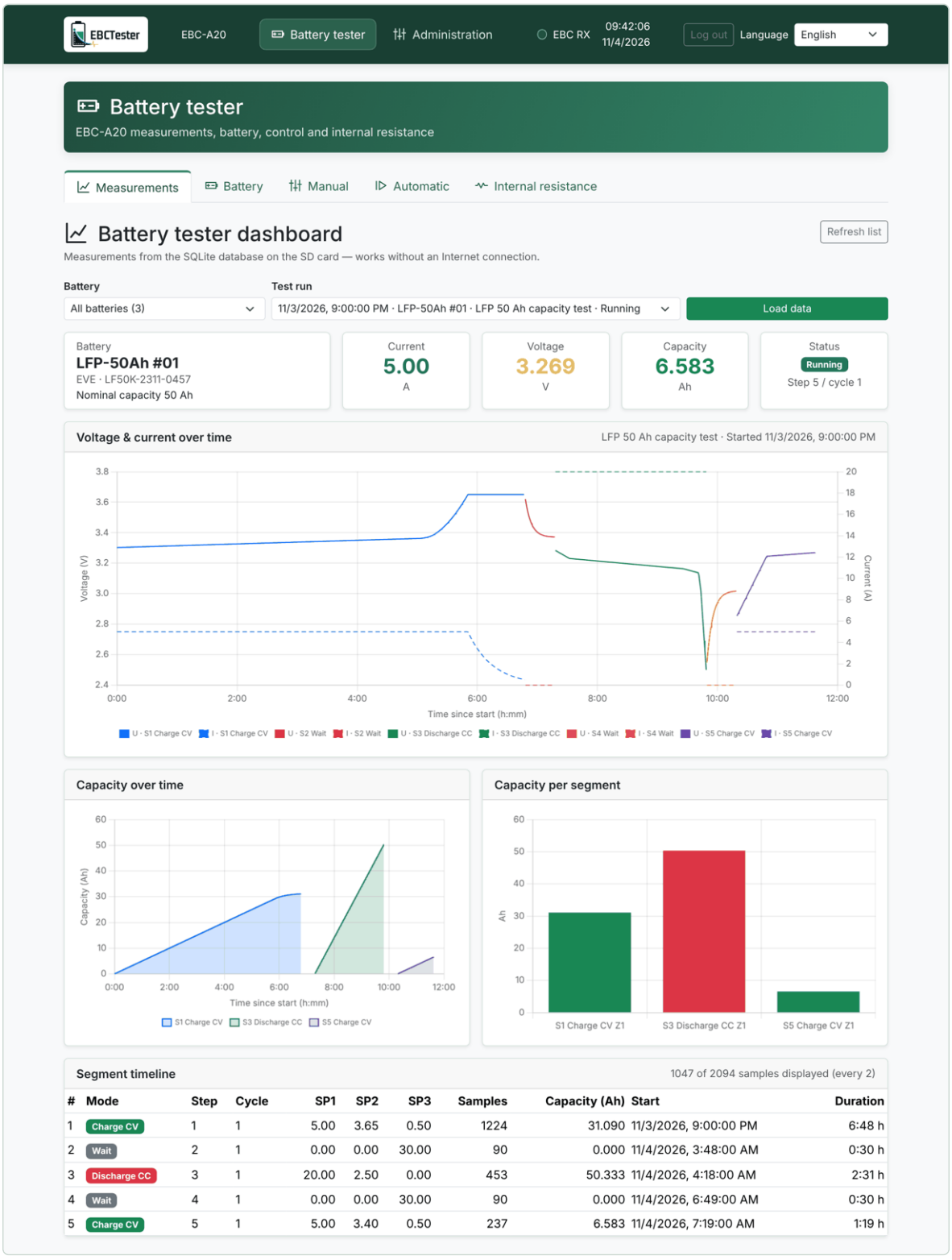


Figure 7: Measurements dashboard during a running capacity test

Selecting a test run

1. Optionally restrict the list with **Battery**.
2. Choose a run under **Test run**. Runs are listed newest first with date, battery, plan and status (or with the custom name given in test-run management).
3. Click **Load data**. **Refresh list** re-reads the list of runs, for example after a new test was started.

The newest run is loaded automatically when the page opens. A running test is refreshed every 15 seconds.

What is shown

- **Summary cards:** battery (label, manufacturer, serial number, nominal capacity), the latest current, voltage and capacity, and the run status with current step and cycle.
- **Voltage & current over time:** one colour per segment; voltage as solid line (left axis), current as dashed line (right axis). The time axis shows hours:minutes since the start of the run.
- **Capacity over time** for every charge or discharge segment, and **Capacity per segment** as a bar chart (green = charge, red = discharge).
- **Segment timeline:** one row per segment with mode, plan step, cycle, the three set-points (SP1–SP3), number of measurements, capacity, start time and duration.

A *segment* is a continuous section of a run with one operating mode – for example "charge CC/CV" or "wait". To keep the charts fast, at most 1500 points are drawn; the line above the timeline states how many of the stored points are shown ("every n-th"). Segment boundaries are always included. Hover over a chart (or tap it) to see the exact values.

Status	Meaning
Running	The test is still being recorded.
Completed	Finished normally (plan complete or cut-off condition reached).
Stopped	Stopped by the user.
Interrupted	Ended by a power loss, restart or lost connection.
Error	Ended because of an error (see the debug log for details).

4.2 Battery

Here you enter the battery that will be tested next. Its data is stored with every test run, so you always know which cell or pack a measurement belongs to.

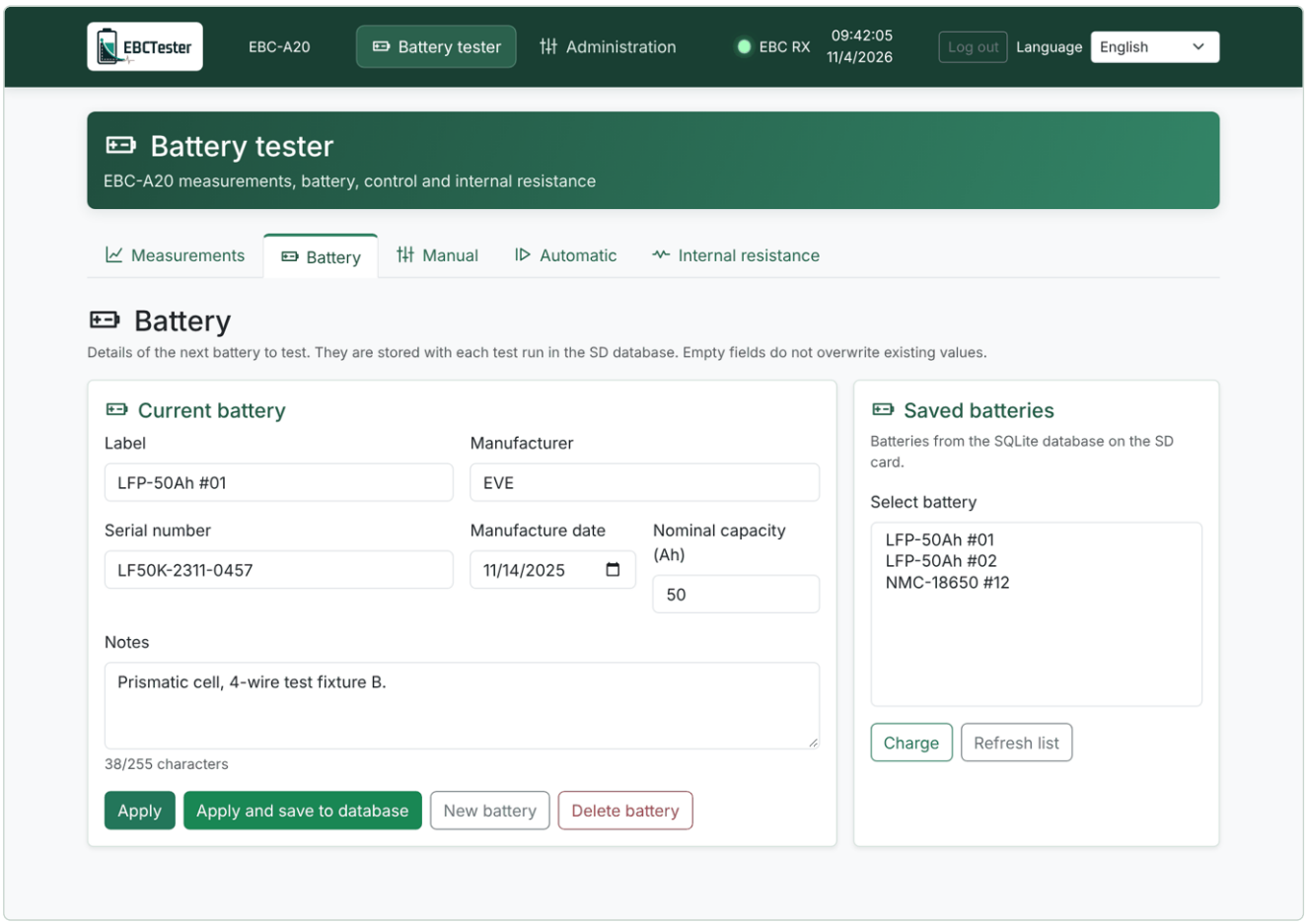


Figure 8: Battery tab

Field	Description
Label	Unique name of the battery, for example *LFP-50Ah #01*. Required for saving to the database.
Manufacturer, Serial number	Free text.
Manufacture date	Date picker.
Nominal capacity (Ah)	Rated capacity; shown in the dashboard for comparison.
Notes	Up to 255 characters.

Button	Function
Apply	Makes the entered battery the current battery (stored on the device). The next test run is recorded with it.
Apply and save to database	Additionally saves the battery in the SD database so it can be selected again later. To rename a saved battery, load it, change **Label**, then click this button.
New battery	Clears the form for a new entry.
Delete battery	Deletes the saved battery shown in the form after confirmation and the login password. Not possible while it is used by recorded runs – delete those runs first in test-run management – or while a recording, plan or DCIR test is active. MariaDB backups are not changed.

Button	Function
Load button / Refresh list	Under **Saved batteries**: select a battery and click the button below the list (labelled **Charge** in firmware 2.0.0 – it loads the battery and does not start charging) or double-click the entry to load it into the form. **Refresh list** re-reads the database.

Empty fields do not overwrite values that are already stored.

4.3 Manual control

The **Manual** tab gives you direct control of the tester – comparable to the buttons on the tester itself, but with recording.

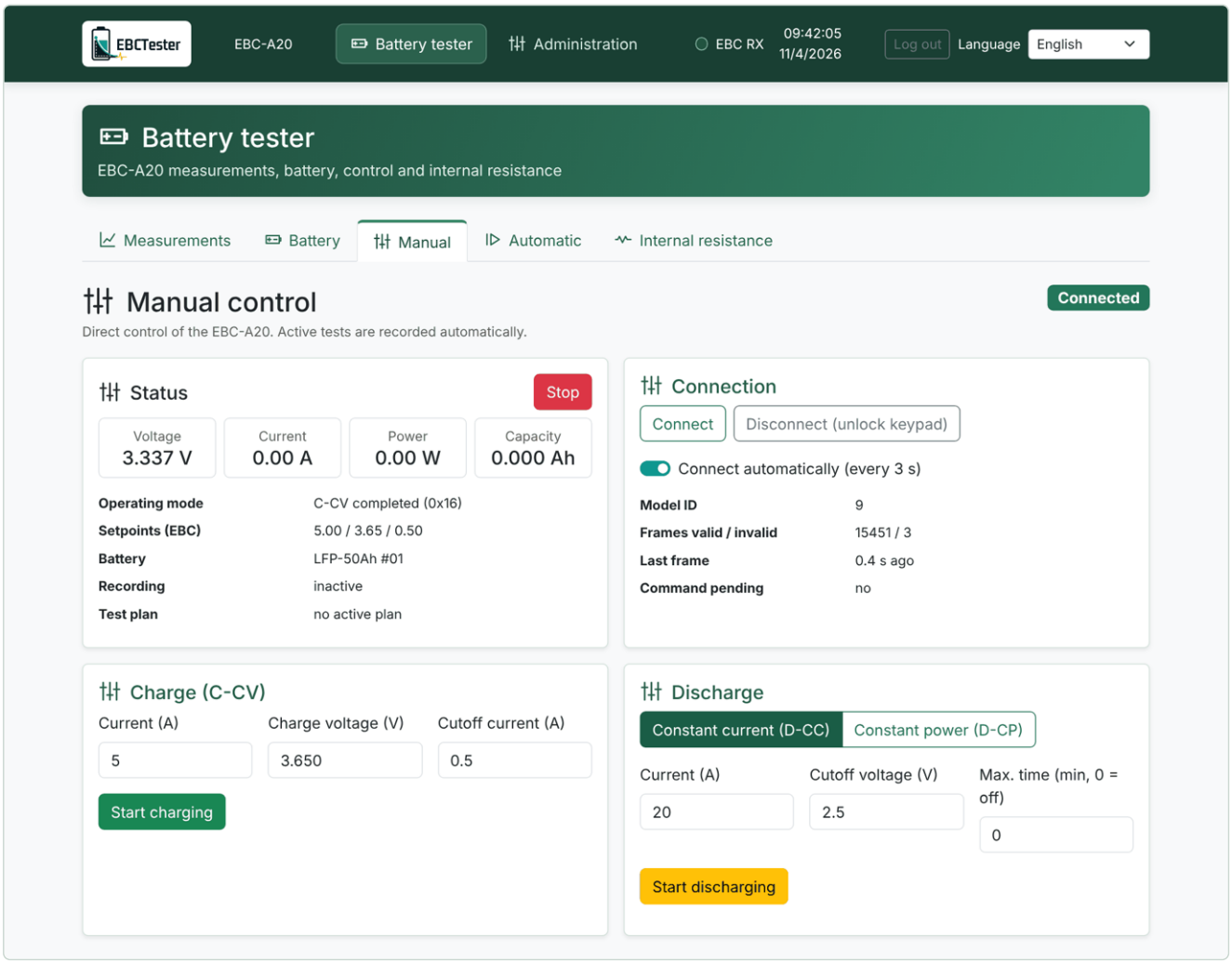


Figure 9: Manual control with the tester connected and idle

Status

Shows the live values from the tester (voltage, current, power, capacity), the operating mode with its code, the set-points currently active in the tester, the current battery, the recording state (number of stored measurements) and whether a test plan is active. The badge at the top right shows *Disconnected*, *Connected* or the active mode. **Stop** stops the current operation immediately; it is always available while connected.

Connection

- **Connect** starts communication with the tester. The tester's keypad is locked while connected.
- **Disconnect (unlock keypad)** ends the connection and unlocks the keypad.

- **Connect automatically (every 3 s)**: EBCTester tries to connect on its own, for example after switching the tester on.
- Diagnostic values: model ID reported by the tester, number of valid/invalid messages, age of the last message, and whether a command is waiting for confirmation.

Charge (C-CV)

Charges with constant current until the charge voltage is reached, then with constant voltage until the current falls below the cut-off current.

Field	Meaning	EBC-A20 range
Current (A)	Charge current in the CC phase	0.01 – 5 A
Charge voltage (V)	End-of-charge voltage (CV level)	up to 18 V
Cutoff current (A)	Charging ends when the current drops below this value	≤ charge current

Discharge

Select **Constant current (D-CC)** or **Constant power (D-CP)**.

Field	Meaning	EBC-A20 range
Current (A) / Power (W)	Discharge current (D-CC) or power (D-CP)	0.01 – 20 A / 1 – 85 W
Cutoff voltage (V)	Discharging ends at this battery voltage	up to 30 V
Max. time (min, 0 = off)	Additional time limit	0 – 239 min

In D-CP mode the power must also be achievable within the maximum current at the cut-off voltage (power ≤ max. current × cut-off voltage). For the A40L/A50L the ranges of the respective model apply.

After **Start charging** or **Start discharging**, EBCTester waits up to 10 seconds for the tester to confirm the mode; otherwise it reports an error and sends Stop. Every manual operation is recorded automatically as a test run.

> **Note**
> Start buttons are disabled while a command is pending, while another operation or a test plan is running (including its wait steps) and while no tester is connected. The reason is shown below the button.

4.4 Automatic control (test plans)

A test plan is a sequence of up to 10 steps that EBCTester executes on its own. The plan runs on the ESP32 and continues without Wi-Fi and without an open browser as long as the device is powered.

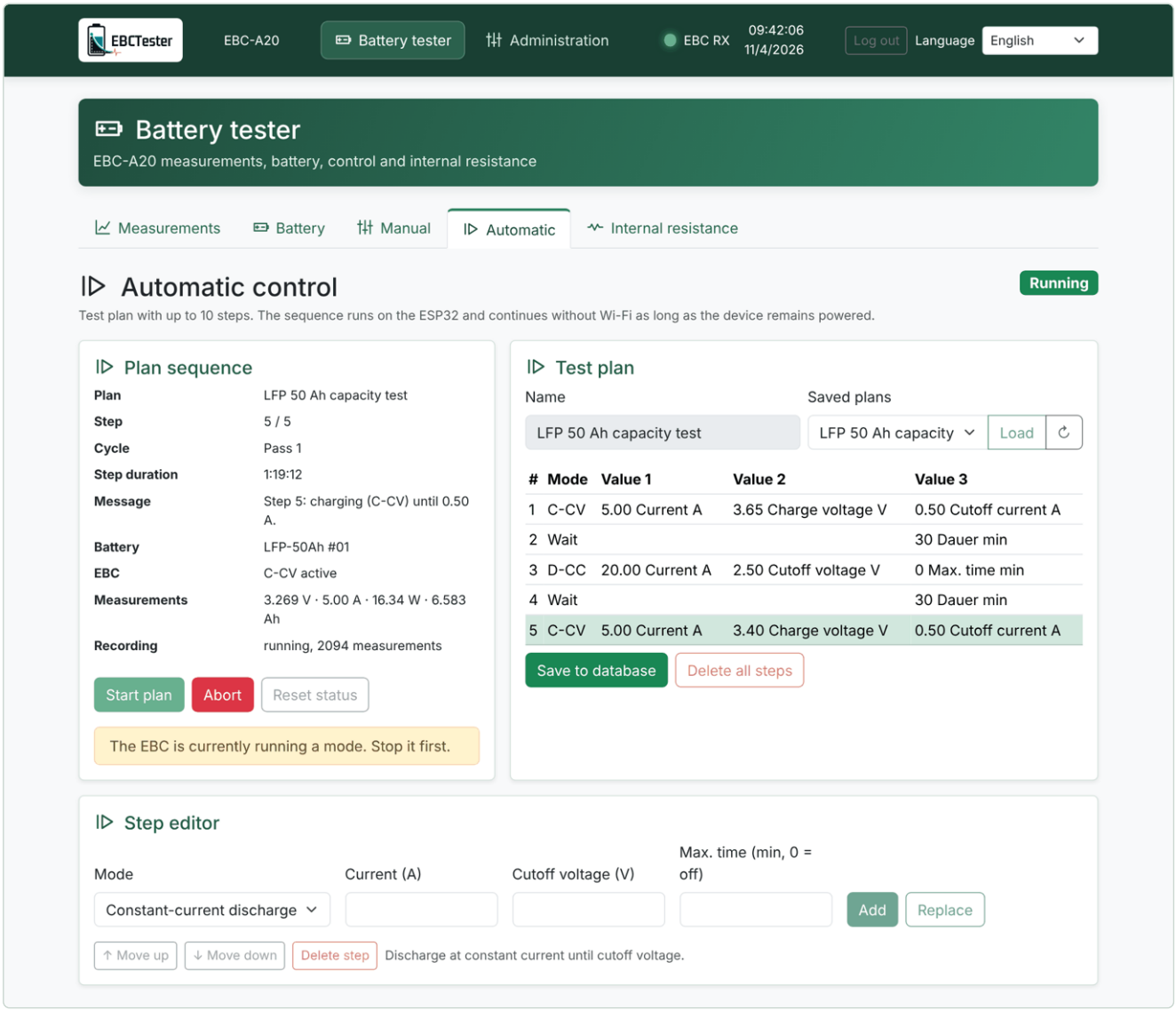


Figure 10: Automatic control: running plan, plan table and step editor

Step types

Mode	Value 1	Value 2	Value 3	Function
D-CC	Current (A)	Cutoff voltage (V)	Max. time (min, 0 = off)	Discharge at constant current until the cut-off voltage.
D-CP	Power (W)	Cutoff voltage (V)	Max. time (min, 0 = off)	Discharge at constant power until the cut-off voltage.
C-CV	Current (A)	Charge voltage (V)	Cutoff current (A)	Charge CC/CV until the cut-off current.
Wait	–	–	Duration (1 – 239 min)	Pause, for example to let the cell recover.
Cycle	–	–	Total passes (1 – 239)	Returns to step 1 until the number of passes is reached. Not allowed as first step and only once per plan.

Creating and editing a plan

1. Enter a **Name** for the plan.

2. In the **Step editor**, choose the mode, fill in the values and click **Add**. The step is appended – or inserted after the selected row.
3. Click a row in the table to select it. You can then **Replace** it with the editor values, move it (**↑ Move up**, **↓ Move down**) or **Delete step**. Click the row again to deselect it.
4. **Save to database** stores the plan under its name on the SD card. **Delete all steps** clears the table.

Every change is saved on the device immediately and checked against the limits of the selected tester. Saved plans can be loaded again with the **Saved plans** list and **Load**; ↻ refreshes the list.

Example: capacity test of a LiFePO4 cell

#	Mode	Values	Purpose
1	C-CV	5.00 A · 3.65 V · 0.50 A	Charge fully
2	Wait	30 min	Let the cell settle
3	D-CC	20.00 A · 2.50 V · 0 (no time limit)	Measure the capacity
4	Wait	30 min	Let the cell recover
5	C-CV	5.00 A · 3.40 V · 0.50 A	Charge to a storage level

Example values only – use the values from your cell's data sheet.

Running a plan

Connect the tester, check the current battery and click **Start plan**. The **Plan sequence** card shows the plan name, current step, cycle, time in the step, status message, battery, tester mode, live values and the number of recorded measurements. The active step is highlighted green in the table.

- **Abort** stops the plan and the tester after confirmation.
- **Reset status** returns a completed, aborted or interrupted plan to "Ready" so it can be started again.

> **Caution**
> After a power failure or restart, a running plan is not resumed automatically. It is marked *Interrupted*. Check the battery, then click **Reset status** and start again if required.

4.5 Internal resistance (DCIR)

This tab measures the DC internal resistance of a battery with a load pulse. The battery rests without load, then a defined discharge current is switched on. From the voltage drop the resistance is calculated three times – by default 1, 5 and 10 seconds after switching on:

$$R = (U_1 - U_2) / I_2$$

U_1 is the average rest voltage at the end of the rest phase, U_2 and I_2 are the voltage and the measured current at the sampling time. Short times mainly reflect the ohmic resistance (contacts, electrolyte); longer times include polarisation effects.

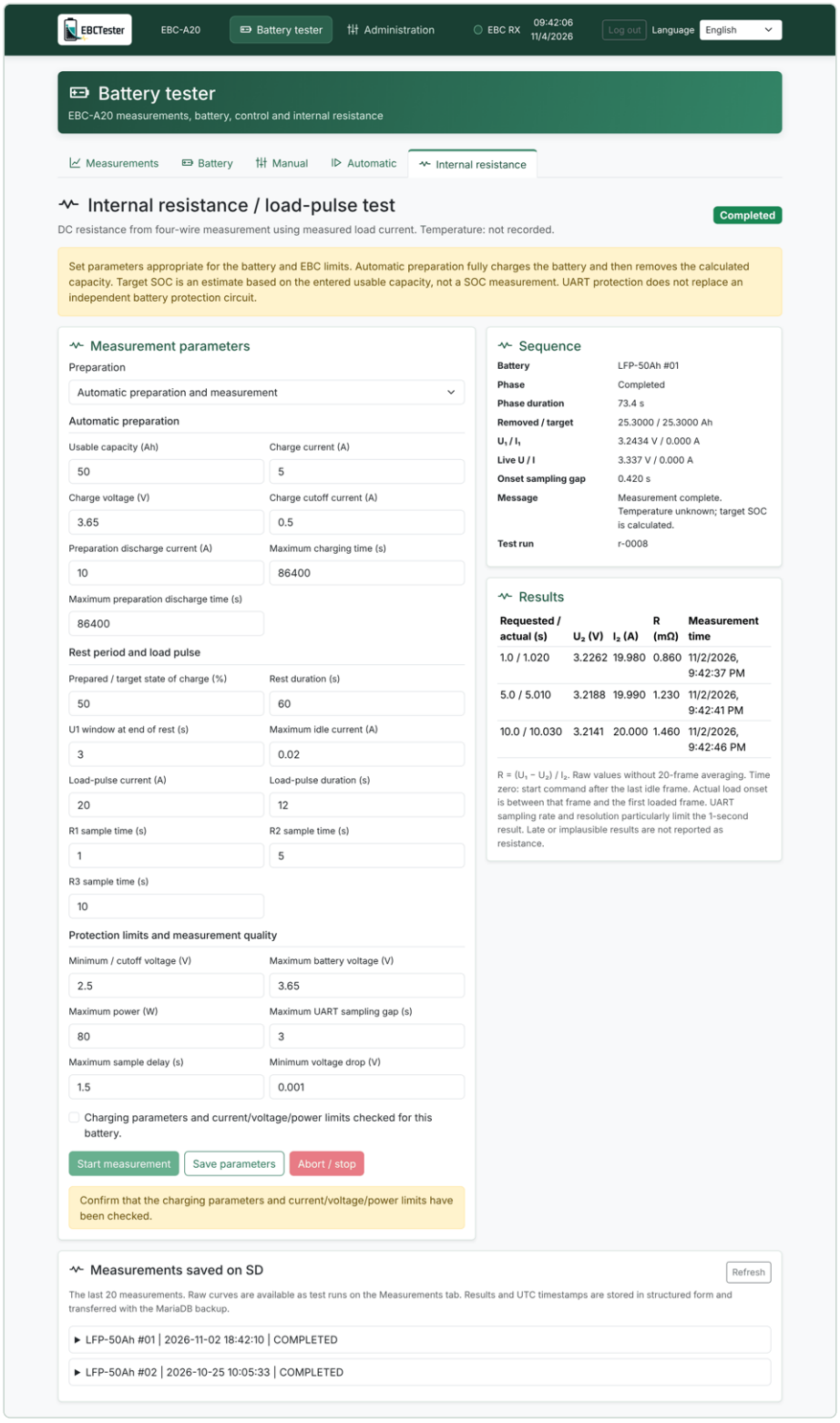


Figure 11: Internal resistance: parameters, sequence and results of a completed test

Preparation methods

- **State of charge already prepared:** the battery is already at the desired state of charge. The test starts with the rest phase.
- **Automatic preparation and measurement:** the battery is first fully charged (C-CV), then the calculated amount of charge is removed: usable capacity × (100 − target SOC) / 100. The removed charge is integrated from the measured current. The target SOC is therefore an estimate based on the entered capacity, not a measured value. A target of 100 % skips the discharge.

Parameters

Group / parameter	Meaning	Default
Automatic preparation (only with automatic preparation)		
Usable capacity (Ah)	Capacity used to calculate the charge to remove	20
Charge current / voltage / cutoff current	Full charge before the measurement	0.5 A / 3.45 V / 0.1 A
Preparation discharge current (A)	Current used to reach the target SOC	0.5
Max. charge / preparation time (s)	Time-outs for both phases	86400
Rest and load pulse		
Prepared / target SOC (%)	Target state of charge (automatic) or information (prepared)	50
Rest time (s)	Rest without load before the pulse	60
U1 window at end of rest (s)	Averaging window for U_1	3
Max. idle current (A)	Current that still counts as "no load"	0.02
Pulse current (A) / duration (s)	Load pulse	0.5 A / 12 s
Sample times R1 / R2 / R3 (s)	Times of the three results after switching on	1 / 5 / 10
Protection limits and measurement quality		
Minimum / cutoff voltage (V)	Test stops below this voltage	2.5
Maximum battery voltage (V)	Test stops above this voltage	3.6
Maximum power (W)	Upper limit for current × max. voltage	50
Max. UART gap (s)	Longest permitted interval between tester messages	3
Max. sample delay (s)	Timing tolerance of the samples	1.5
Min. voltage drop (V)	Smaller drops are rejected as not credible	0.001

Consistency rules: sample times must increase; the pulse must last at least as long as the last sample time plus the timing tolerance; the rest time must be at least the U1 window; cut-off and charge voltage must be below the maximum voltage; and current × maximum voltage must not exceed the maximum power. In addition, all values must be within the limits of the selected tester.

Running the measurement

1. Connect the battery with four-wire sensing (separate sense leads) – otherwise the cable resistance is included in the result.
2. Check all parameters and click **Save parameters**.
3. Tick the confirmation box ("Charging parameters and current/voltage/power limits checked for this battery"). Changing any parameter clears this box again.
4. Click **Start measurement**. The SD database must be ready.
5. Follow the progress in the **Sequence** card; **Abort / stop** ends the test at any time.

The **Results** table lists for each sampling time the requested and actual time, U_2, I_2, the resistance in mΩ and the time stamp. Late or missing samples, too small or negative voltage drops or missing current produce an "invalid" result instead of a wrong number. *Onset sampling gap* shows the uncertainty of the switch-on moment caused by the interval between tester messages – this mainly limits the accuracy of the 1-second value.

Every DCIR test is stored as a normal test run (raw curve in Measurements) plus structured results. **Measurements saved on SD** lists the last 20 DCIR tests. Temperature is not measured and is reported as unknown.

> **Note**
> During a DCIR test all other starts, connect/disconnect and battery changes are blocked. Stop and Abort always remain available. A DCIR test is never resumed after a restart.

5 Administration section

5.1 Device information

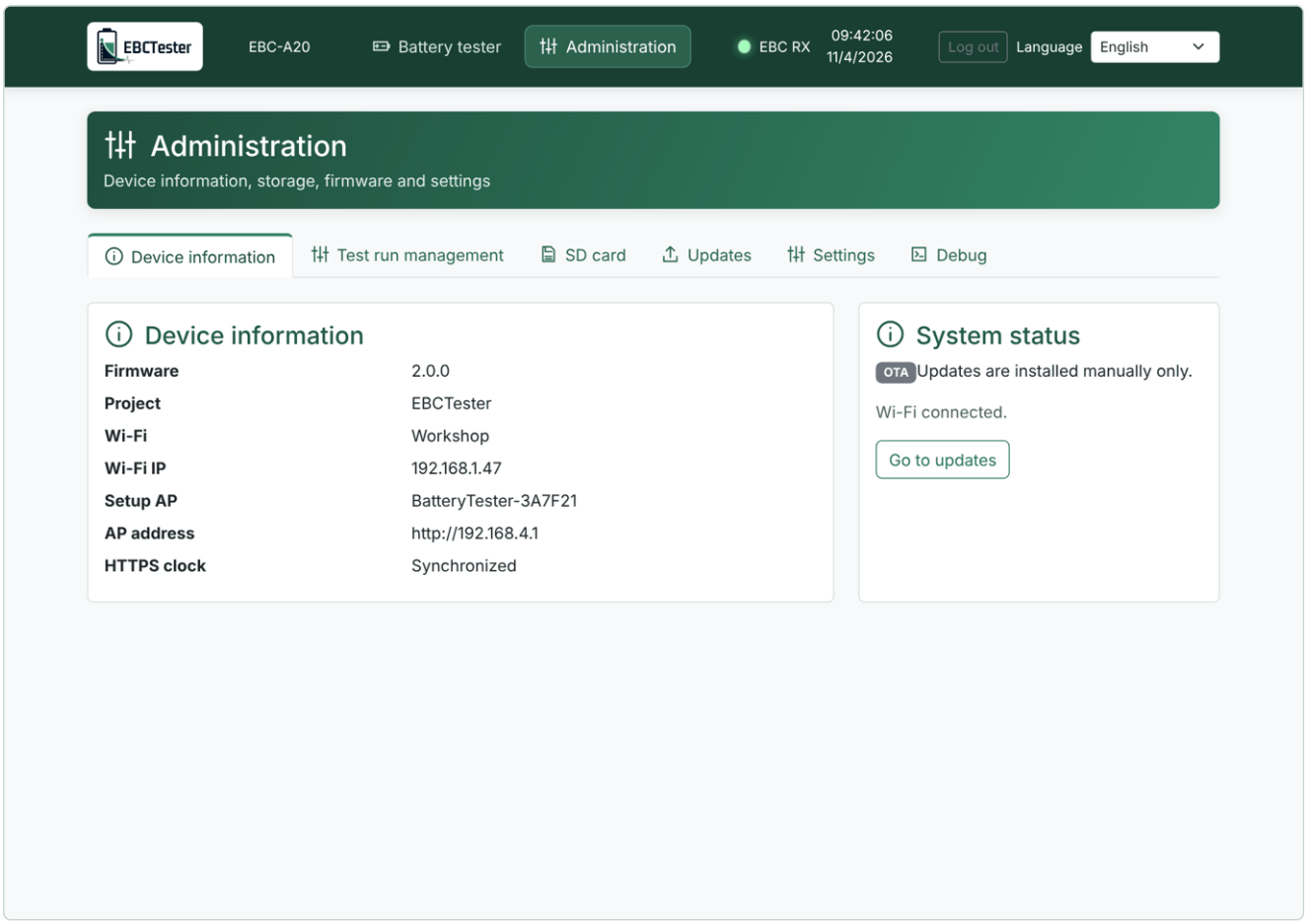


Figure 12: Device information

Shows the installed firmware version, the project name, the connected Wi-Fi network and IP address, the name of the setup access point (`BatteryTester-XXXXXX`), its address `http://192.168.4.1` , and whether a valid time is available (*HTTPS clock*; set by NTP or the real-time clock). **System status** shows the network message and leads to the update page.

5.2 Test run management

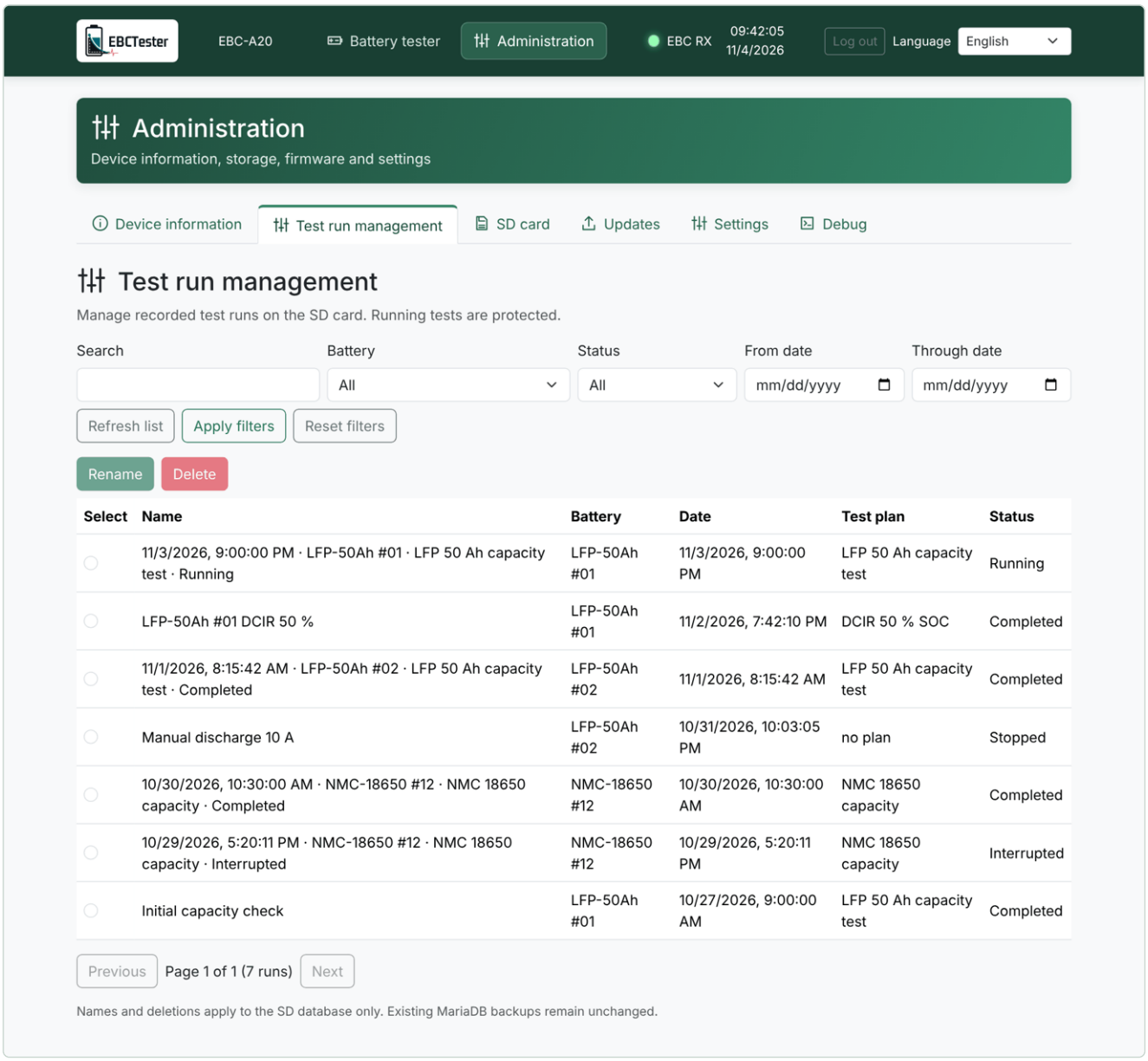


Figure 13: Test run management

This table lists all runs in the SD database, 50 per page, newest first.

- **Filter:** free-text **Search** (name, battery, plan, date, status or run ID), **Battery**, **Status** and a date range (**From date** / **Through date**; the end date includes the whole day). Click **Apply filters**; **Reset filters** clears them.
- **Rename:** select one run with the radio button and click **Rename**. Up to 100 characters; an empty name restores the automatic name. The name is also used in the dashboard.
- **Delete:** select a run and click **Delete**. After confirmation the run is permanently removed together with its measurements, segments and DCIR results. Batteries and plans are kept.

Running runs are protected and cannot be renamed or deleted. Renaming and deleting require the login password and only affect the SD database – existing MariaDB backups are not changed, and custom names are not transferred to MariaDB.

> **Caution**
> Deleting cannot be undone. Make a backup (MariaDB synchronisation or a copy of the database file) before deleting important runs, and do not delete runs while a MariaDB synchronisation is running.

5.3 SD card

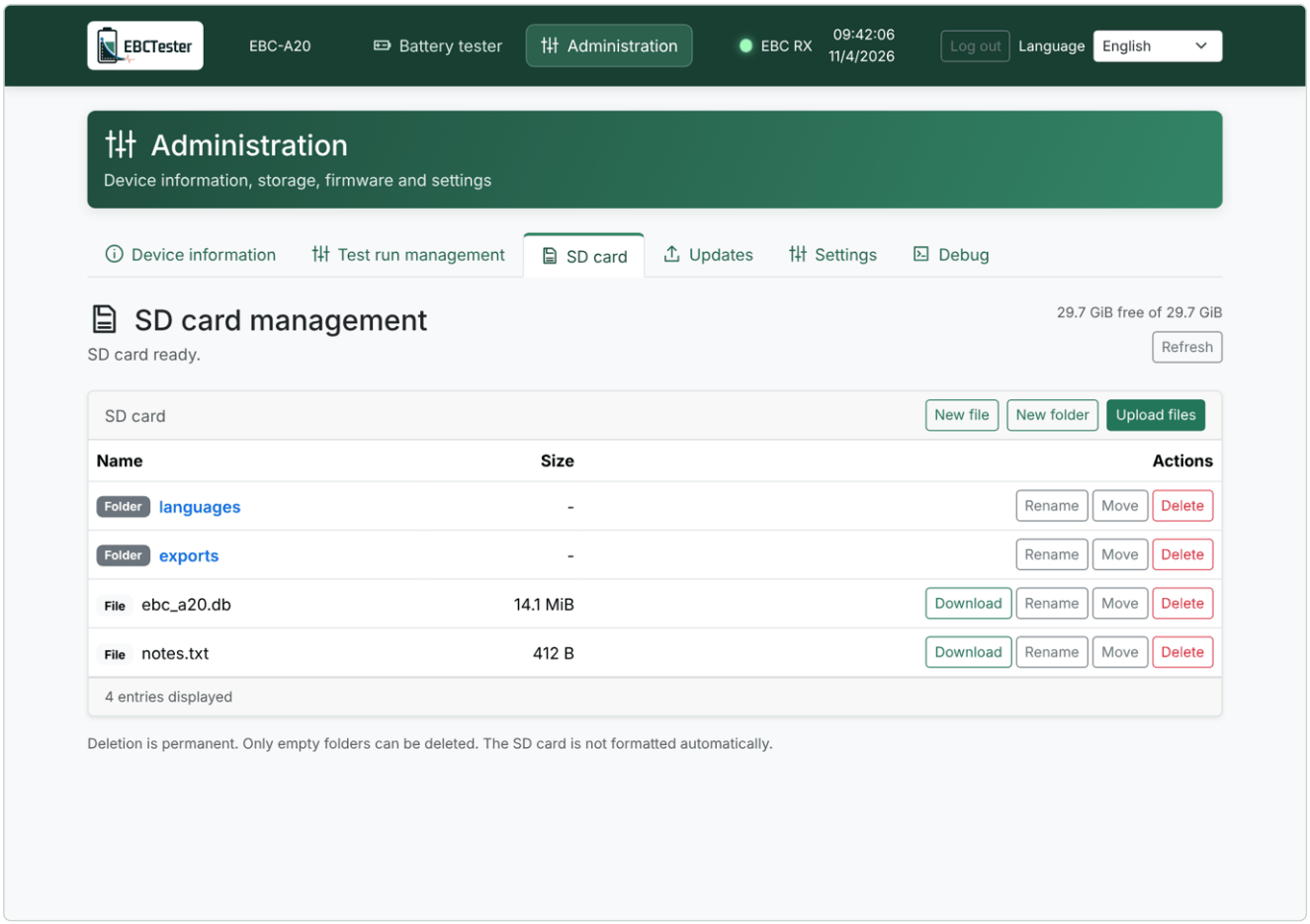


Figure 14: SD card management

A simple file manager for the microSD card – no need to remove it from the device. The header shows whether a card is mounted and how much space is free.

- Click a folder name to open it; use the breadcrumb path to go back.
- **New file**, **New folder** and **Upload files** (several files possible, with progress bar).
- For each entry: **Download** (files only), **Rename**, **Move** and **Delete**.

Deleting is permanent; only empty folders can be deleted. The card is never formatted automatically. The measurement database is the file `ebc_a20.db` in the root folder; download it regularly as a backup.

> **Caution**
> Do not rename, move or delete the database file while the device is in use. Download it only when no test is being recorded to get a consistent copy.

5.4 Updates

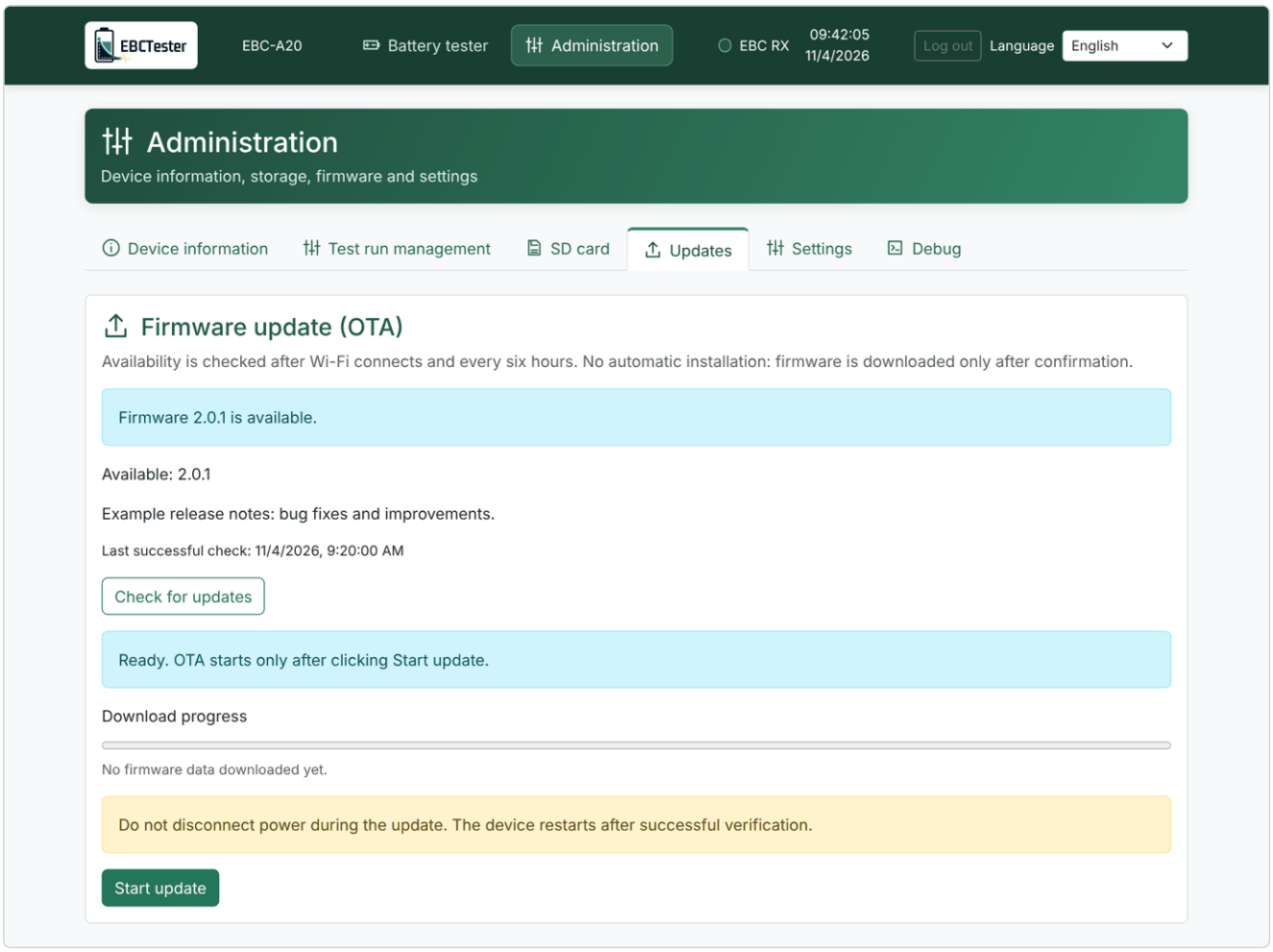


Figure 15: Firmware update

When the device is connected to the Internet it checks for new firmware after connecting, every six hours and once each time you open the web interface. Only a small description file is downloaded – firmware is never installed automatically.

1. If an update is available, the new version and its release notes are shown. **Check for updates** repeats the check manually.
2. Make sure no test is running, then click **Start update** and confirm the version.
3. The firmware is downloaded into the inactive memory area and checked (size and SHA-256 checksum). Progress is shown.
4. After successful verification the device restarts. The new firmware is only accepted permanently if it runs stable for 10 seconds; otherwise the previous version is restored automatically.

> **Warning**
> Do not disconnect the power during an update, and never update during a battery test.

A firmware update keeps all settings, including the Maker configuration, the passwords and the Wi-Fi credentials.

While the configuration is inactive, update checks are switched off; the tab is still shown.

5.5 Settings

The **Settings** tab collects all device settings. When you save a setting you are asked for the current login password (labelled **Admin password** in the dialog) as an extra protection – also when browser login is switched off. Cancelling or entering a wrong password does not save anything. The access-point password is not used here. On Maker devices this confirmation can be switched off in Maker Settings.

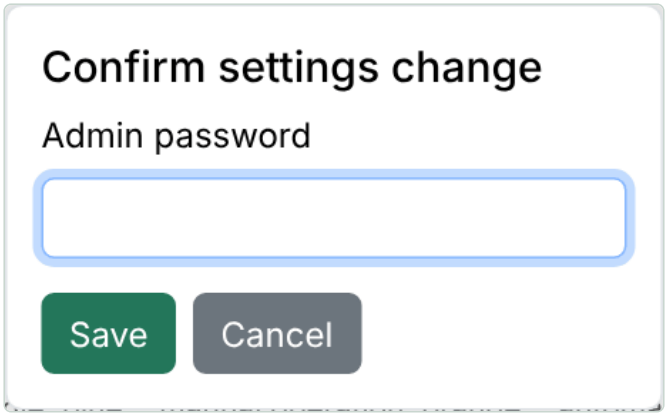


Figure 16: Password confirmation when saving settings

Login Settings

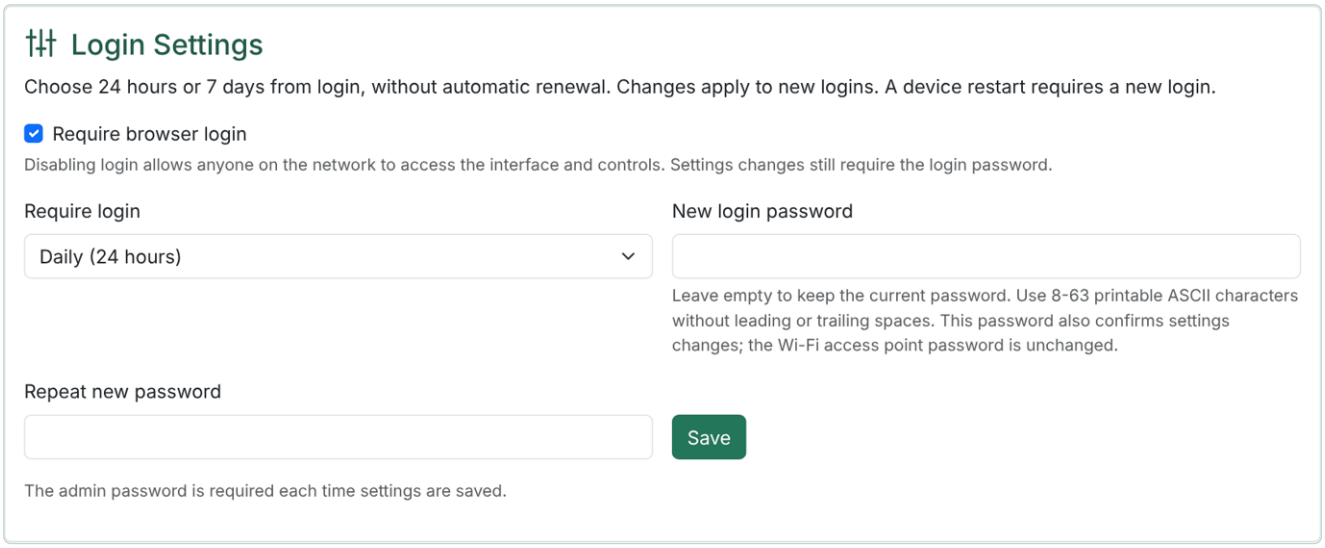


Figure 17: Login Settings

Setting	Description
Require browser login	On by default. When switched off, the web interface opens without logging in and the **Log out** button disappears.
Require login	**Daily (24 hours)** (default) or **Weekly (7 days)**. The duration counts from the login, is not extended by activity and applies to new logins. Only available while browser login is on.
New login password / Repeat new password	Enter the new password twice. 8–63 printable ASCII characters (letters, digits and common symbols, no umlauts), no spaces at the beginning or end. Leave both fields empty to keep the current password.

Click **Save** and confirm with the current login password – also when you are changing the password itself. The new password is stored permanently on the device and survives restarts and firmware updates. The access-point password is not changed. Changing the password or switching browser login on or off ends all existing sessions; the page reloads automatically. Afterwards log in with the new password.

> **Warning**
> With browser login switched off, everyone in the network can view all data and operate the tester – including starting and stopping tests. Use this only in a network you trust completely. Settings changes still require the login password.

Configure Wi-Fi

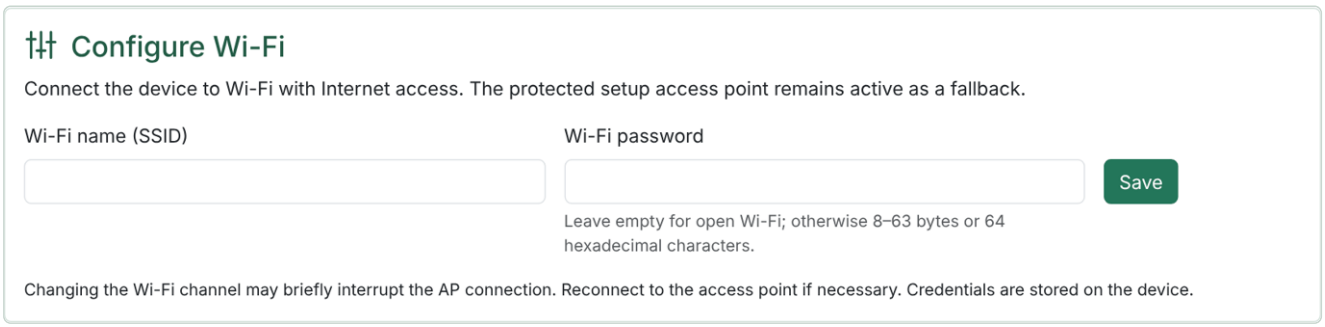


Figure 18: Wi-Fi settings

Network name (SSID) and password of your home network. Leave the password empty for an open network; otherwise 8–63 characters or 64 hexadecimal characters. The credentials are stored on the device.

EBC tester

Selection of the tester model – see Selecting the tester model (section 2.6).

Home Assistant MQTT

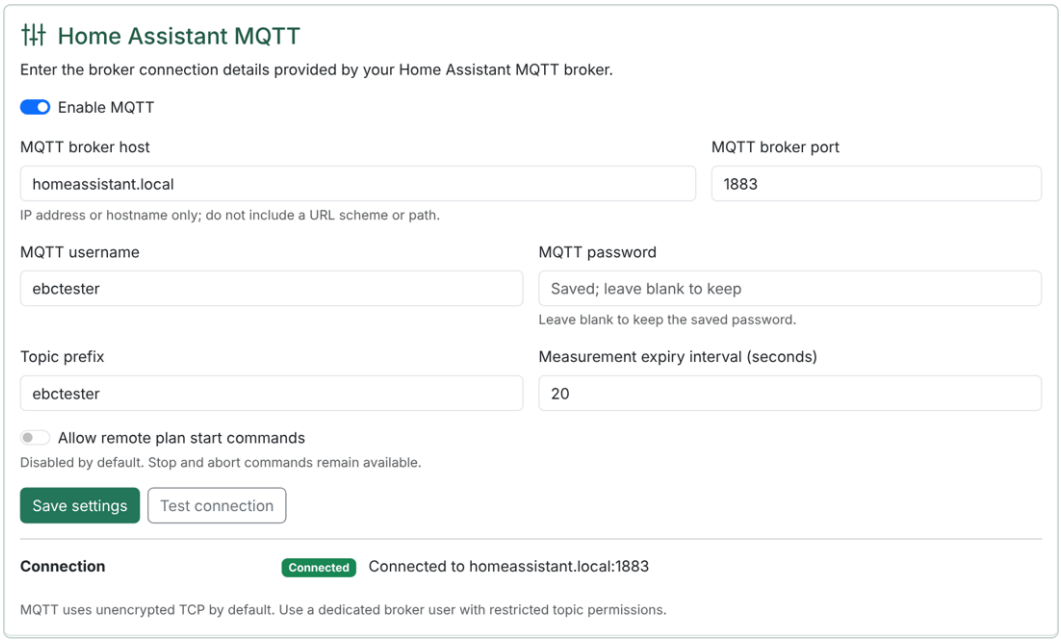


Figure 19: Home Assistant MQTT

Optional connection to an MQTT broker, typically the one used by Home Assistant. The device then appears automatically in Home Assistant (MQTT discovery) and publishes voltage, current and capacity of every recorded measurement.

Enable MQTT	Switches the client on or off.
Broker host / port	IP address or host name (without `mqtt://`), port usually 1883.
User name / password	Broker account. Leave the password empty to keep the saved one.
Topic prefix	Start of all topics, for example `ebctester`.
Measurement expiry interval (s)	Interval between recorded measurements; values become "unavailable" after three intervals without new data.
Allow remote plan start commands	Off by default. Only when enabled can a plan be started via MQTT. Stop and Abort always work.

Save settings applies the settings immediately, **Test connection** reconnects with the saved settings. The connection state is shown below. MQTT is unencrypted – use a dedicated broker user with restricted rights. Details are given in Home Assistant MQTT topics (section 15.2).

MariaDB backup

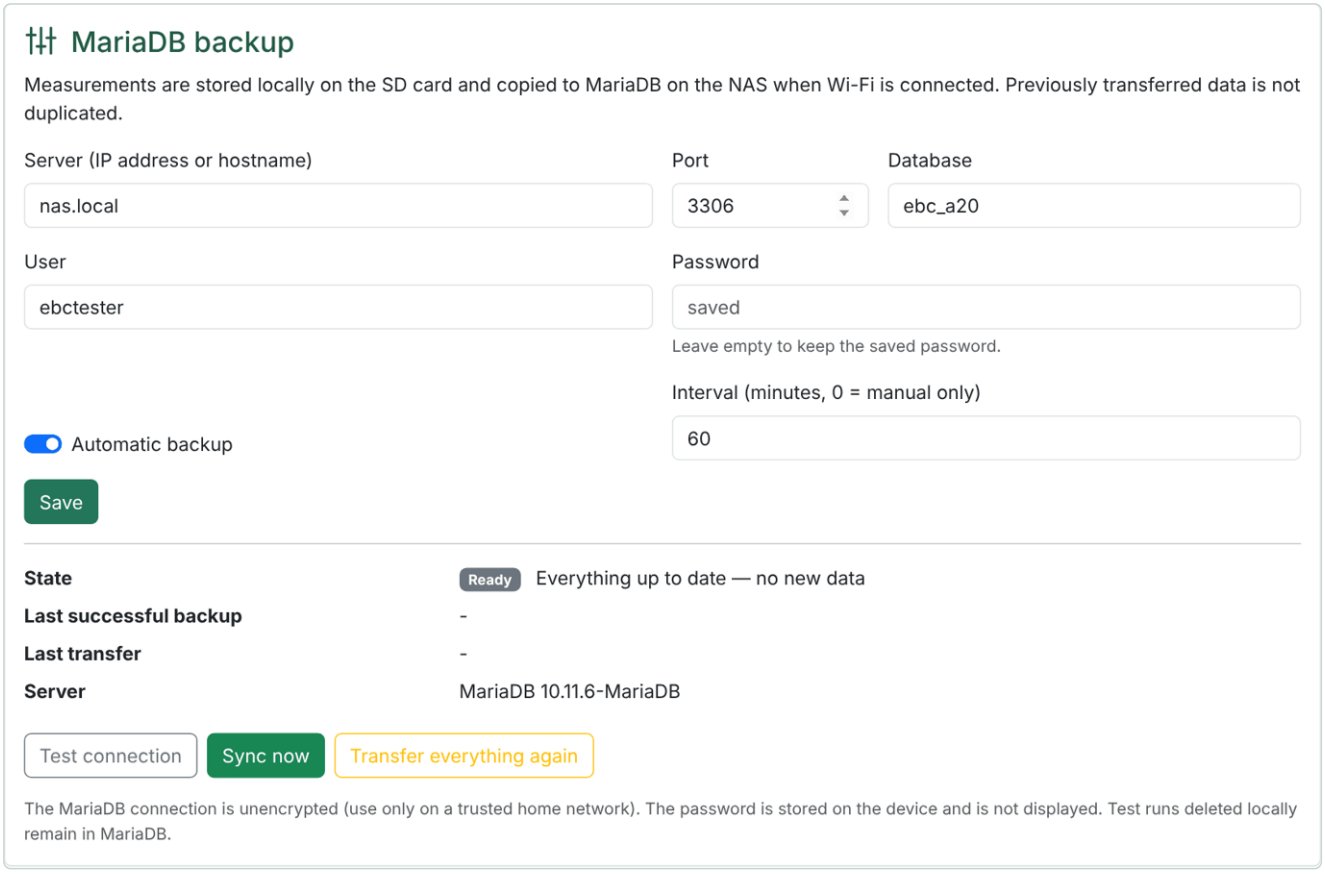


Figure 20: MariaDB backup

Copies the recorded test runs to a MariaDB database, for example on a NAS. Only new data is transferred; repeated or interrupted synchronisations do not create duplicates.

- Enter server, port (3306), database, user and password; choose **Automatic backup** and an interval in minutes (0 = manual only). Save.

- **Test connection** checks access, **Sync now** starts a transfer immediately, **Transfer everything again** re-sends all data (no duplicates are created).
- The status area shows state, last successful backup, number of runs and measurements transferred, and the server version.

Automatic backup only runs with Wi-Fi, a ready SD card and a saved password. Runs deleted on the device remain in MariaDB. The database tables must already exist on the server – see MariaDB backup database (section 14.2). The connection is unencrypted: use it only in your home network.

LED settings

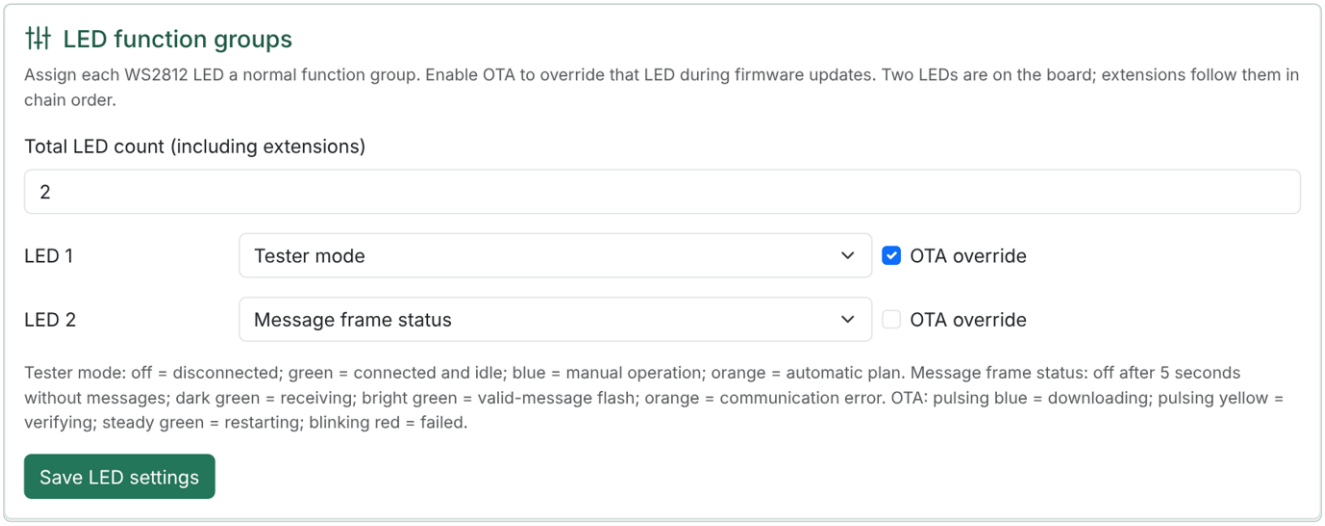


Figure 21: LED function groups (as delivered)

Here you decide what each WS2812 status LED shows. The LEDs are numbered in chain order, starting at the LED connected to the configured GPIO.

Total LED count	Number of LEDs in the chain, 1–16, including extensions. One row per LED appears below.
LED 1 … LED n	Function group: **Tester mode**, **Message frame status** or **Off**. Several LEDs may show the same group. The colours are explained in LEDs (section 2.8).
OTA override	During a firmware update this LED shows the update state instead of its function group.

Save LED settings applies the assignment immediately (after password confirmation) and keeps it across restarts and firmware updates. Defaults: two LEDs; LED 1 = Tester mode with OTA override, LED 2 = Message frame status. If the status LED is disabled in Maker Settings or the configuration is inactive, a note is shown; the assignment is stored and used as soon as the LEDs are enabled.

> **Note**
> Every WS2812 draws current from the 5 V supply (up to about 60 mA at full white; EBCTester uses low brightness). With USB-only supply keep extensions small.

Debug log

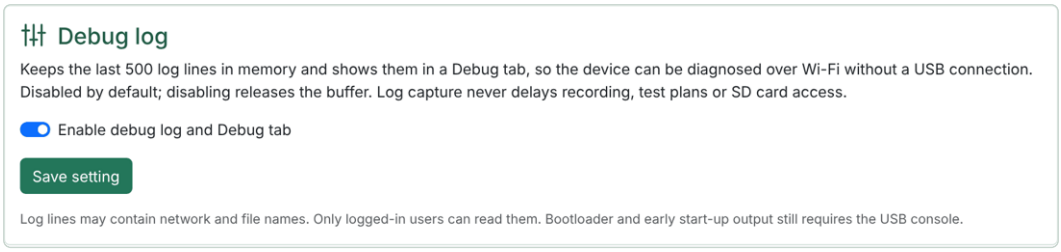


Figure 22: Debug log setting

When enabled, the device keeps the last 500 log lines in memory and shows them in the **Debug** tab. Disabled by default; disabling frees the memory. Logging never delays recording, plans or SD access.

Language



Figure 23: Language

English and German are built in. Additional languages can be added as JSON files in the folder `/languages` on the SD card (see Appendix A). **Reload languages** reads the folder again; then select the language in the header.

Device reset

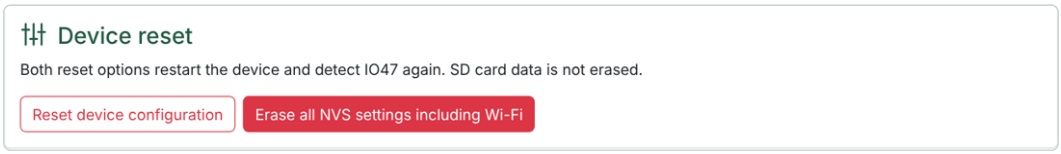


Figure 24: Device reset

Two reset functions return the device to the state of its first start. Both ask for confirmation and the login password, restart the device and detect the mode (GPIO 47) again. Data on the SD card – measurements, batteries, plans and language files – is never erased.

Button	Resets	Keeps
Reset device configuration	Device configuration (all Maker Settings and ConfigAktive), login password (back to `ebctester`), login options, access-point password, Firmware mode	Wi-Fi credentials, tester model, current battery and plan, MQTT and MariaDB settings, debug-log setting, LED settings, the time in the real-time clock
Erase all NVS settings including Wi-Fi	Everything stored on the device, including Wi-Fi credentials, tester model, LED settings and all service settings	Only the firmware and the SD card

A reset is refused while a test, plan or recording is running or a firmware update is in progress. Stop the test and wait until the recording has finished.

> **Note**
> After a reset, Maker devices start inactive again with the default pin assignment of the firmware. Note your Maker Settings before resetting.

5.6 Maker Settings

On an active Maker device, **Maker Settings** is the last tab of the Administration section. It contains the same form as during the first setup (see Maker Settings reference (chapter 3)). Every save restarts the device.

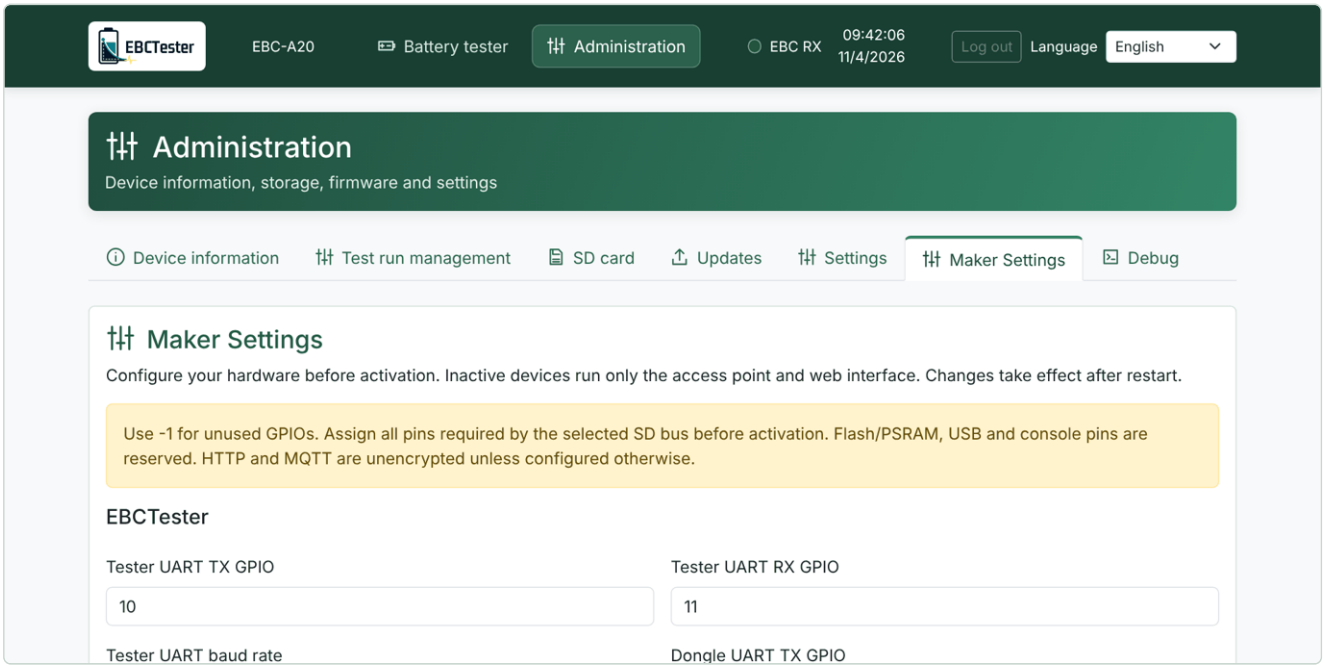


Figure 25: Maker Settings on an active device

5.7 Debug

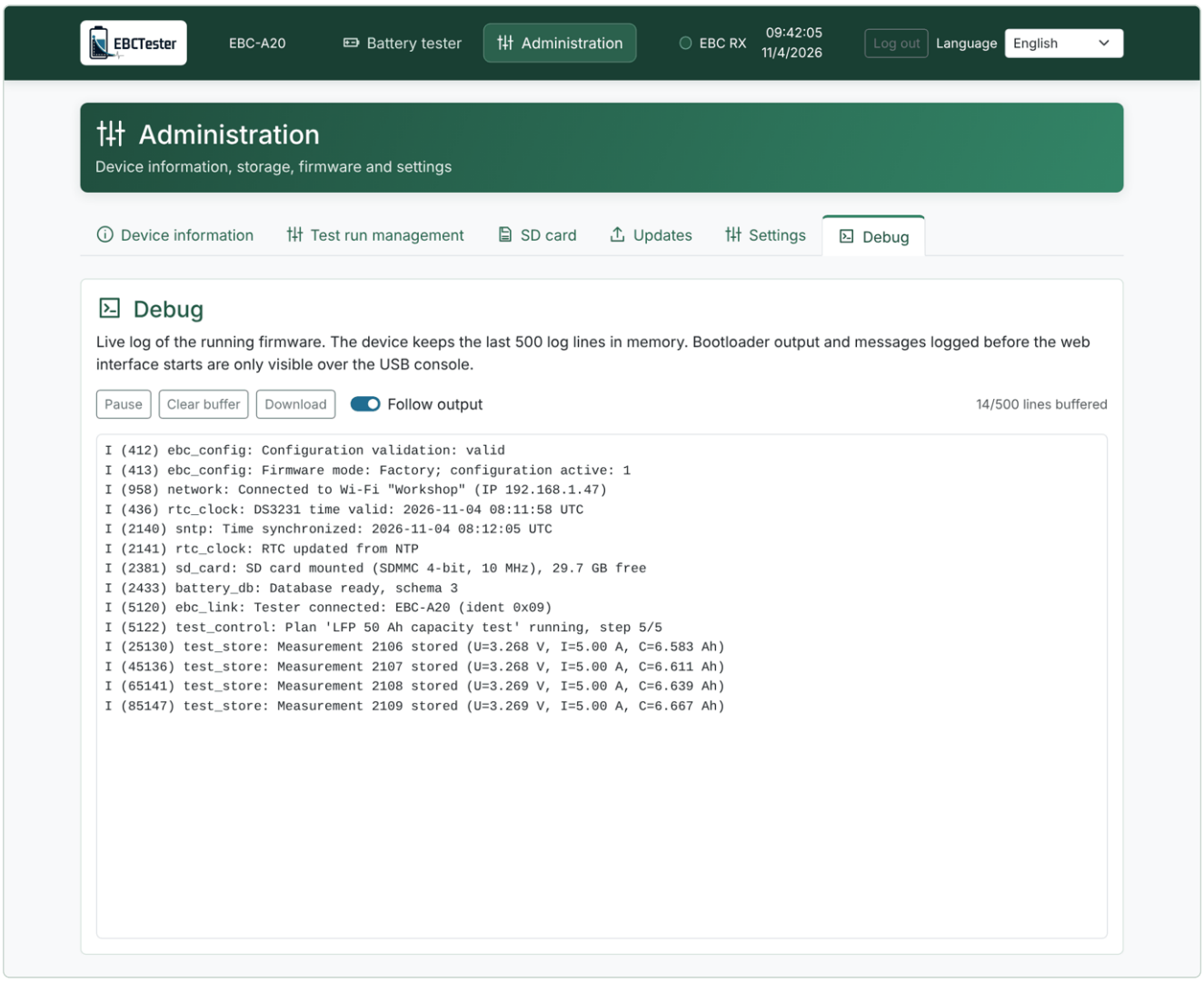


Figure 26: Debug tab

Live log of the running firmware – useful when something does not work and no USB cable is at hand. **Pause** freezes the display, **Clear buffer** empties it, **Download** saves the lines as a text file, and **Follow output** keeps the view at the newest line. Log lines may contain network and file names, but never passwords. Messages from the very beginning of the start-up – including the mode detection – are only visible on the USB console.

6 Typical workflows

6.1 Measuring the capacity of a battery

1. **Battery:** enter the battery data and click **Apply and save to database** (or load a saved battery and click **Apply**).
2. **Plan:** in **Automatic**, load or create a plan: charge (C-CV) – wait – discharge (D-CC) – wait – optionally charge to storage level.
3. **Connect:** in **Manual**, click **Connect** (or enable automatic connection). The RX indicator in the header turns green.
4. **Start:** in **Automatic**, click **Start plan**. You can now close the browser.
5. **Follow:** open **Measurements** at any time. The capacity of the discharge segment in the segment timeline is the measured capacity.
6. **Finish:** when the plan is complete, the run status changes to *Completed*. Give the run a meaningful name in **Test run management** if you like.

6.2 Measuring internal resistance at 50 % state of charge

1. Apply the battery as above and connect it with four-wire sensing.
2. In **Internal resistance**, select **Automatic preparation and measurement**, enter the usable capacity (ideally the result of a capacity test), the charge parameters and target SOC = 50 %.
3. Set the pulse current (for example 0.2 C to 1 C, within the battery's limits), check the protection limits, save, tick the confirmation and start.
4. After completion compare the three results. Repeat measurements under the same conditions (temperature, SOC, rest time) to compare batteries.

6.3 Backing up your data

- **Simple:** download `ebc_a20.db` in **SD card** when no test is running. The file can be opened with any SQLite tool (for example "DB Browser for SQLite").
- **Automatic:** set up the MariaDB backup. Delete runs on the SD card only after verifying that they have been transferred.

6.4 Changing the hardware of a Maker device

1. Stop all tests and wait until the recording has finished.
2. Open **Maker Settings**, untick **ConfigAktive** and click **Save and restart**. The device restarts without driving any hardware pin.
3. Switch the device off and change the wiring.
4. Switch it on, enter the new pin assignment, tick ConfigAktive and save again.

7 Troubleshooting

Problem	Possible cause	Remedy
Access point `BatteryTester-…` not visible	No antenna, device not powered or still starting	Check the antenna and power; wait 10–20 s after power-on.
AP password unknown	Randomly generated at first start, or changed in Maker Settings	Read a generated password from the start-up log over USB (serial terminal, press RESET). Otherwise use **Erase all settings over USB** below.
Login password not accepted	Password was changed; or the default is still active	Try `ebctester` (default until a password is saved, and after a device reset). A forgotten password can only be reset over USB, see below.
Page does not load at 192.168.4.1	Phone switched back to mobile data or another network	Disable mobile data while connected to the AP; check the Wi-Fi connection.
Login keeps returning to the login page	Session expired or device restarted; browser blocks cookies	Log in again; allow cookies for the device address.
Login refused although the password is correct	Maximum of 8 simultaneous sessions reached	Log out in other browsers or restart the device.
Battery tester section missing; only Settings and Device information shown	Configuration inactive (ConfigAktive off, or a stored configuration failed the pin checks)	Open Maker Settings, check the pins, tick ConfigAktive and save. The debug log or USB console shows the reason.
"Invalid configuration" when saving Maker Settings	Pin used twice, reserved or not output-capable; required pin set to −1; password too short	Check the rules in Activation and validation (section 3.5).
Device starts in FactoryMode although it is a Maker build	GPIO 47 is pulled high by the circuit	Leave GPIO 47 open or tie it to GND, then reset the device configuration.
RX indicator stays grey	Tester not connected or wiring wrong	Click **Connect**; check TX/RX crossing and GND; check that the tester is switched on. Check the tester UART pins in Maker Settings.
"Tester mismatch" error	Selected model differs from the connected one	Select the right model in **Settings → EBC tester**, then **Connect**.
Start buttons disabled	See the yellow hint below the button	Typical: not connected, waiting for first message, plan running, command pending.
"Database not ready"	No card, card not FAT32, card full or defective, wrong SD pins or bus mode	Insert a FAT32 card; check the **SD card** tab. exFAT cards must be reformatted to FAT32 on a PC.
Times shown as uptime instead of date; header clock shows dashes	No valid time: no Internet and the real-time clock is disabled, not connected or not set yet	Connect the device to Wi-Fi with Internet access for a few minutes; this also sets the real-time clock. Check the RTC pins and the pull-ups; the debug log shows the RTC messages.
Header time differs from local time	The browser converts UTC into its own time zone	Check the time-zone setting of the phone or PC. The stored data is always UTC.

Problem	Possible cause	Remedy
Status LED dark	Function group *Off*, tester not connected (Tester mode) or no messages (Message frame status); LED disabled, wrong GPIO or configuration inactive	Check the assignment in **Settings → LEDs** and the LED pin in Maker Settings.
Message-frame LED orange	Invalid messages, wrong tester model selected, or a model that can only be read	Check wiring and the model selection in **Settings → EBC tester**.
Plan shows "Interrupted"	Power failure or restart during the plan	Check the battery, click **Reset status**, restart the plan if needed.
DCIR result "invalid"	Sample too late, voltage drop too small, no current	Increase the pulse current, check four-wire sensing and the timing tolerance.
Update check fails	No Internet, clock not yet synchronised, server unreachable	Wait a few minutes after connecting; retry with **Check for updates**.
Settings are not saved	Login password wrong or dialog cancelled	Enter the current login password (not the access-point password) in the confirmation dialog.
Reset or configuration change refused	A test, plan, recording or update is running	Stop the test and wait until the recording has finished.
Language file not offered	Wrong folder, invalid JSON or wrong code	Place it in `/languages`, check the warning shown in the header, click **Reload languages**.

Erase all settings over USB

If neither the login password nor the access-point password is known, the settings memory can be erased with a PC. Firmware and SD card data are kept; all settings return to the state of the first start:

```
esptool.py --chip esp32s3 --port COM5 erase_region 0x9000 0x6000
```

Replace `COM5` with the serial port of the device. If the port is not found, start the device in download mode (hold BOOT, press and release RESET, release BOOT) and press RESET after erasing.

If the problem persists, enable the debug log, reproduce the problem and download the log. When reporting a problem on GitHub, include your hardware and Maker Settings, the firmware version, steps to reproduce and the log – but never passwords or private data.

Part II – Technical Reference

This part is intended for readers who build the hardware themselves, compile the firmware, integrate EBCTester into other systems or want to understand how it works internally. It covers the start-up and configuration logic, the reference circuit, connectors and parts, firmware configuration, the tester protocol, data storage, interfaces and security.

8 System architecture

8.1 Firmware structure

The firmware is written in C/C++ with Espressif's ESP-IDF 6.0.1 for the ESP32-S3. It is divided into functional components; the main program only initialises services. Networking, updates and SD-card handling are separate, reusable repositories included as Git submodules under `external/`.

Repository / component	Responsibility
ebc_config	Central device configuration in NVS, GPIO 47 mode detection, validation, reset functions
main	Start-up sequence; Maker Settings tab and device reset (ebc_config_web); DS3231 driver and time synchronisation (rtc_clock, ds3231_time); LED function groups (led_policy, led_settings)
ebc_link	UARTs, frame parser, model detection and communication with the tester
test_control / test_store	Manual control, test plans, batteries, recording of test runs
resistance_test	DCIR preparation, load pulse and evaluation
sqlite3 / battery_database	SQLite adapter, SD file-system layer and database schema
battery_dashboard	Measurements tab, charts (Chart.js) and test-run management
mariadb_sync	Incremental backup to MariaDB
ha_mqtt	Optional Home Assistant MQTT integration
ui_localization / debug_log	Additional SD languages; in-memory log and Debug tab
ota-suite	Wi-Fi provisioning, access point, web server and shared UI, login, manual OTA updates, status LED
sd-card-manager	SDMMC/SPI driver, card detection, shared storage lock, file-management tab

8.2 Data flow

The tester sends a status message roughly once per second. `ebc_link` checks each message (checksum, model identifier) and passes it to the controller. While a test is active, `test_store` averages a configurable number of messages (default 20, about one stored measurement every 20 seconds) and writes the measurement in a database transaction to the SD card. Plans and DCIR tests are state machines running in their own tasks; they react to the received messages and send commands to the tester. The web interface only reads data and sends commands through authenticated HTTP requests – it is never needed for a test to continue.

9 Start-up and device configuration

9.1 Start-up sequence

Before any other service starts, the firmware loads the device configuration from the non-volatile storage (NVS) of the ESP32. Only on a new device, or if the stored configuration is not valid, does it create a new one and detect the firmware mode:

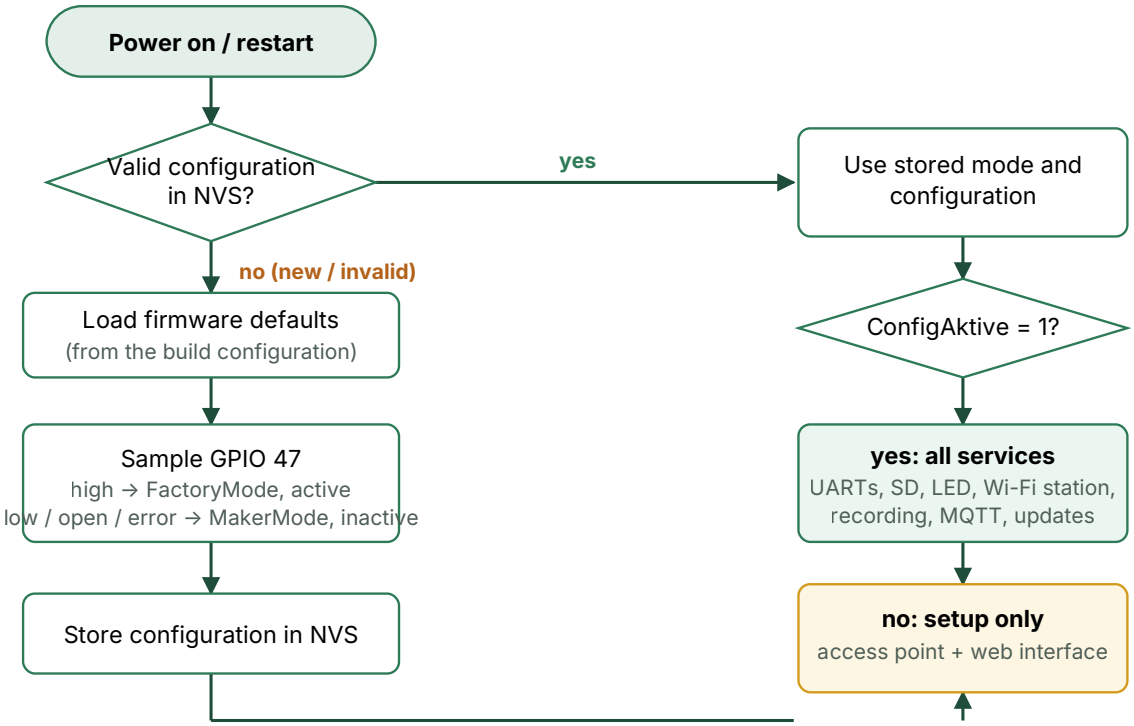


Figure 27: Start-up sequence and mode selection

A valid stored configuration always takes precedence: GPIO 47 is evaluated only when a new configuration is created – on a new device, after a device reset or when the stored data is damaged.

9.2 Firmware modes

	FactoryMode	MakerMode
Selected by	GPIO 47 high at the first start	GPIO 47 low, open or inconsistent at the first start
Configuration at first start	Active (ConfigAktive = 1)	Inactive (ConfigAktive = 0)
Hardware configuration	Firmware defaults (pin assignment of the original board); not editable in the browser	Firmware defaults as a proposal; edited in **Maker Settings**
Maker Settings tab	not present	present
Settings-change confirmation	always on	can be switched off in Maker Settings
Access-point password	firmware default	editable in Maker Settings

	FactoryMode	MakerMode
Real-time clock	always enabled, GPIO 14/15/16 enforced	optional, off by default, pins editable

While the configuration is inactive, no tester or dongle UART, SD card, status LED, I²C bus or RTC, Wi-Fi station connection, time synchronisation, update check, recorder or MQTT client is started. Only the access point and the web interface run. The flash memory, the PSRAM and the USB console of the ESP32 are outside this guarantee.

9.3 GPIO 47 detection

The level of GPIO 47 is determined without loading the pin permanently:

1. The internal pull-up is switched on; after 200 µs the pin is read.
2. The internal pull-down is switched on; after 200 µs the pin is read again.
3. Steps 1–2 are repeated three times. Afterwards both pull resistors are switched off and the pin is disabled, so no current flows through the internal resistor during operation.

Read with pull-up	Read with pull-down	Result	Mode
1	0	open (pin follows the internal resistor)	MakerMode
0	0	low (tied to GND)	MakerMode
1	1	high (tied to 3.3 V)	FactoryMode
0	1	inconsistent (interference)	MakerMode, warning in the log

The pull-up is "1" only if it is 1 in all three passes; the pull-down result is "1" if it is 1 in any pass. An external connection must be clearly stronger than the internal resistors (about 45 kΩ): use a direct connection or a resistor of at most 10 kΩ. On the original board, solder jumper JP1 ties GPIO 47 directly to 3.3 V (pads 1–2, as delivered) or to GND (pads 2–3).

9.4 Configuration storage

All hardware-related settings are stored in the NVS namespace `ebc_cfg`. After the first start the firmware uses only these stored values; the build configuration (sdkconfig) only supplies the defaults for a new configuration.

Key	Type	Content
`config`	blob	Header (magic `0x45424332` "EBC2", structure version 2, payload length, CRC-32) followed by the complete configuration structure
`FirmwareMode`	u8	0 = MakerMode, 1 = FactoryMode
`ConfigAktive`	u8	0 = inactive (setup only), 1 = active

The configuration is written in the order blob – FirmwareMode – ConfigAktive and then committed. The blob is written as a whole: after a power failure during writing either the old or the new version is present.

Valid version-1 configurations are CRC-checked and migrated to version 2 without resetting existing values, mode or activation. RTC is enabled in FactoryMode, including existing version-2 installations. MakerMode preserves a saved RTC selection and defaults to off, including migration from version 1.

Validity check

A stored configuration is valid only if the namespace exists, the blob has the expected size, magic and structure version, the CRC-32 over the payload is correct, `FirmwareMode` exists and the content passes the general checks of Activation and validation (section 3.5). The start-up log reports the result: *new*, *new or invalid size/magic*, *version mismatch*, *CRC mismatch* or *valid*. A missing or invalid `ConfigAktive` starts the device inactive; an active configuration that fails the pin checks also starts inactive.

If `nvs_flash_init()` reports a full or incompatible NVS partition (for example after an interrupted erase), the partition is erased and initialised again; the device then starts like a new one.

Stored values

Group	Values
tester	tester UART TX/RX/baud, dongle UART TX/RX/baud, averaging length, device ID, MQTT enabled
ota	AP password, SNTP server, manifest URL, web (login) password, primary tabs, tab order, HTTP OTA allowed, status LED enabled and GPIO, settings-password confirmation, SD languages, session duration (1/7 days), browser login enabled
sd	bus (0 = 4-bit SDMMC, 1 = SPI), SDMMC and SPI frequency, card-detect GPIO, mount point, CLK/CMD/D0–D3, MISO/MOSI/CS/CLK
rtc	RTC enabled, INT GPIO (reserved input), I²C SDA and SCL GPIO (structure version 2). FactoryMode forces enabled with GPIO 14/15/16 on every load and save.

Other NVS namespaces

Namespace	Content	Reset device configuration
`ebc_cfg`	Device configuration, mode, ConfigAktive	erased
`network`	Wi-Fi station credentials	kept
`tester`	Tester model, current battery, plan, manual values, DCIR parameters	kept
`ha_mqtt`	Broker settings	kept
`debug_log`	Debug-log setting	kept
`leds`	LED count and function groups (versioned blob)	kept
others	MariaDB settings, Wi-Fi driver data, OTA state	kept

Erase all NVS settings including Wi-Fi calls `nvs_flash_erase()` for the whole partition. Both resets end with `esp_restart()`; the next start runs the complete first-start sequence.

9.5 Firmware defaults

A new configuration takes these values from the firmware build. The release firmware uses the pin assignment of the original board, so that it works in FactoryMode without any setting. On a Maker device they are only a proposal in Maker Settings.

Setting	Default	Setting	Default
Tester UART TX / RX	GPIO 10 / 11, 9600 Bd	**SD bus**	4-bit SDMMC, 10 MHz
Dongle UART TX / RX	GPIO 8 / 9, 9600 Bd	**SD CLK / CMD**	GPIO 6 / 7
Averaging length	20 frames	**SD D0 / D1 / D2 / D3**	GPIO 5 / 4 / 18 / 17
Device ID	`EBC-A20`	**Card detect**	GPIO 13 (low = card inserted)
Status LED	on, GPIO 48	**Mount point**	`/sdcard`
Access-point password	`ebctester`	**Login password**	`ebctester`, 24 h, login required
SNTP server	`pool.ntp.org`	**Settings confirmation**	on
MQTT	off	**HTTP OTA**	off
Real-time clock	INT / SDA / SCL = GPIO 14 / 15 / 16; Factory: on, Maker: off	**LED functions**	2 LEDs: tester mode + OTA, message frames

Caution

Firmware 2.0.0 evaluates the card-detect input as active-high only. The switch of the original board is active-low (GPIO 13 reads low with a card inserted). On a Maker device with this circuit, enter `-1` for the card-detect GPIO until a polarity option is available.

10 Hardware

The EBCTester board is a compact controller board. It contains no measurement circuit of its own – all measurement and power electronics are in the ZKE tester. The board provides the microcontroller, power supply, SD card, real-time clock and a level-shifted serial interface.

Microcontroller	ESP32-S3-WROOM-1U-N16R8: dual-core 240 MHz, 16 MB flash, 8 MB octal PSRAM, Wi-Fi 2.4 GHz with u.FL antenna connector
Power supply	19 V via J4 (EBC-A20 adapter) or 5 V via USB-C, both may be connected
Storage	microSD socket, 4-bit SD mode, card-detect switch
Real-time clock	DS3231 with CR2032 backup cell, always used in FactoryMode
Tester interface	2 × UART, 5 V TTL via TXS0104E level shifter
Indicators	Power LED, 2 × WS2812B RGB LED (configurable function groups), chain extension J5
Board size	80 × 80 mm, 4 × M3 mounting holes (connected to GND)
Revision	Schematic revision 1 (2 November 2026), PCB marking "EBCTester V1.0", date code 20261103

10.1 Block diagram

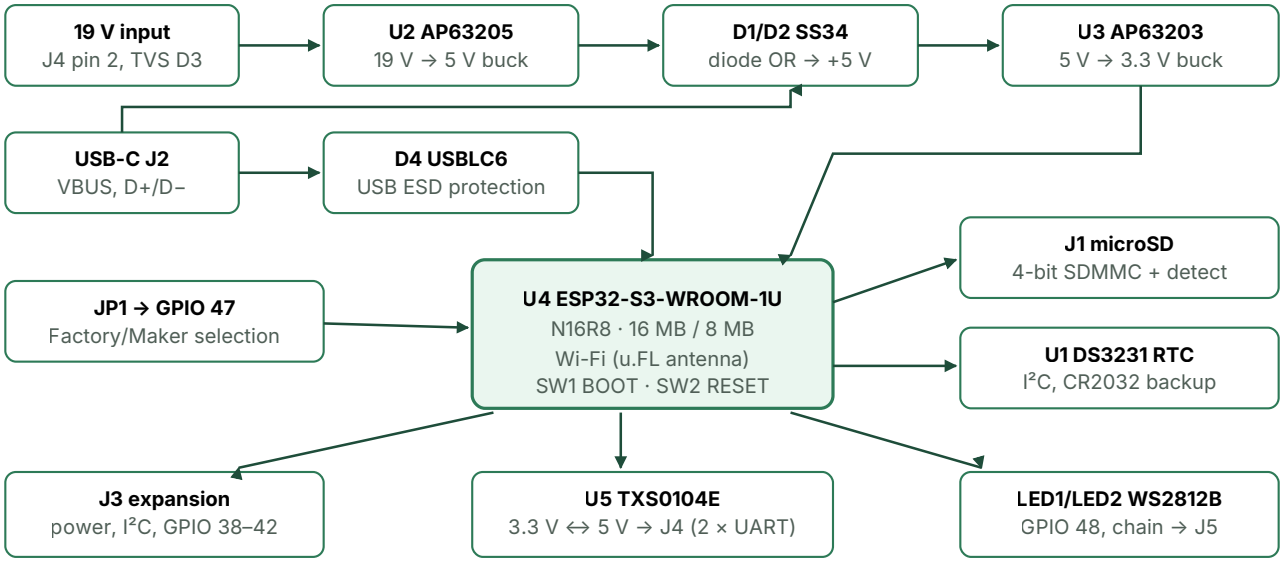


Figure 28: Block diagram of the EBCTester board

10.2 Circuit description

Power supply

The 19 V input (from the EBC-A20 power supply via J4) is protected by the unidirectional TVS diode D3 (SMBJ20A, cathode to +19 V; stand-off voltage 20 V, clamping voltage 32.4 V – below the 35 V absolute maximum input of U2). D3 also protects against reverse polarity by conducting, so the supply should be current-limited or fused. The synchronous buck converter U2 (AP63205, fixed 5 V, up to 2 A) with L1 (4.7 µH)

produces 5 V. This output and the USB supply (VBUS) are combined by the Schottky diodes D1 and D2 (SS34) into the +5 V rail, which is about 4.6–4.7 V after the diode drop. The board can therefore run from the 19 V input, from USB, or from both at the same time. With USB only, the total current is limited to the 500 mA of the USB port.

A second buck converter U3 (AP63203, fixed 3.3 V) with L2 supplies the ESP32-S3 and all logic from the +5 V rail, so 3.3 V is available from either source. Input and output capacitors follow the Diodes Inc. data sheet: 10 µF/50 V at the 19 V side, 2 × 22 µF at each output, 100 nF bootstrap capacitors. The data sheet suggests 3.9 µH for the 3.3 V version; 4.7 µH is within the permitted range (2.2–10 µH).

Microcontroller and USB

U4 is an ESP32-S3-WROOM-1U-N16R8 module with 16 MB flash and 8 MB octal PSRAM. The "U" variant has no PCB antenna but a u.FL connector – an external 2.4 GHz antenna is mandatory. GPIO 35–37 are used internally by the octal PSRAM and must not be used. The USB-C connector J2 is wired to the native USB of the ESP32-S3 (GPIO 19/20) for flashing and the console; D4 (USBLC6-2SC6) protects the data lines against ESD. R10/R11 (5.1 kΩ on CC1/CC2) identify the board as a USB device to USB-C power supplies.

SW1 (BOOT, GPIO 0, pull-up R12) and SW2 (RESET, EN, pull-up R13 with C16 for a defined power-on reset) allow manual download mode: hold BOOT, press and release RESET, release BOOT.

microSD card

J1 (Hirose DM3AT) is connected in 4-bit SD mode. All data and command lines have 10 kΩ pull-ups (R1–R5) as required by the SD specification. The card-detect switch is normally open and closes when a card is inserted; with pull-up R6, the signal SD_CD reads low with a card present.

Real-time clock

U1 (DS3231SN) is a temperature-compensated RTC (±2 ppm) on I²C with 4.7 kΩ pull-ups (R7/R8). Its INT/SQW output (open drain, pull-up R9) goes to GPIO 14 and can be used as an interrupt or deep-sleep wake-up source. A non-rechargeable CR2032 cell (BT1) on VBAT keeps the time when the board is unpowered; the cell voltage can be measured at TP11. The firmware accesses the DS3231 at address 0x68 with 100 kHz and 100 ms time-out. It validates the calendar data (BCD, 12/24-hour format, month lengths, leap years, no date before 2024) and rejects the time while the oscillator-stop flag is set; the flag is cleared only after NTP time has been written. The oscillator is enabled for battery operation, square-wave and alarm outputs are switched off. INT is reserved; alarms and wake-up are not implemented. RST and 32KHZ are not connected.

Serial interface

The tester uses 5 V TTL levels, but ESP32-S3 inputs are not 5 V tolerant. U5 (TXS0104E) is a 4-bit bidirectional, auto-direction level shifter between 3.3 V (VCCA) and the +5 V rail (VCCB). It carries UART1 (tester) and UART2 (reserved for the PC dongle) to connector J4. The TXS0104E is suited to push-pull signals at the low baud rate used (9600 Bd).

Status LEDs

LED1 and LED2 are WS2812B RGB LEDs with integrated driver, connected in a chain to GPIO 48 through the series resistor R15 (470 Ω). Each LED has a 100 nF decoupling capacitor (C17, C18). The WS2812B runs on the +5 V rail, but the ESP32 drives the data line with 3.3 V. To keep this level above the input-high threshold of LED1 (0.7 × VDD), LED1 is supplied through the Schottky diode D5, which lowers its supply by about 0.3 V. LED2 receives the regenerated 5 V signal from LED1. The data output of LED2 is led through R16 (470 Ω) to connector J5, so that further WS2812B LEDs can be added to the chain. The firmware drives up to 16 LEDs in this chain; their function is set in **Settings → LEDs**.

Power-OK LED and mode jumper

D6 is connected to +3.3 V through R14 (470 Ω) and lights up as soon as the 3.3 V supply is present – independently of the firmware. The three-pad solder jumper JP1 ties GPIO 47 to +3.3 V (pads 1–2, bridged as delivered) or to GND (pads 2–3) and selects the firmware mode at the first start.

10.3 Connectors

J4 – tester connector (JST XH, 6-pin, 2.5 mm)

Pin	Signal	Description
1	GND	Ground
2	VIN_19V	19 V supply input (from the EBC-A20 power adapter)
3	UART1_TX_5V	Data from EBCTester to the tester → tester RX
4	UART1_RX_5V	Data from the tester → tester TX
5	UART2_TX_5V	Reserved for the PC dongle bridge
6	UART2_RX_5V	Reserved for the PC dongle bridge

TX/RX are named from the ESP32's point of view. UART2 is initialised by the firmware but the bridge between the manufacturer's PC software and the tester is not implemented yet.

J3 – expansion connector (JST XH, 14-pin, 2.5 mm)

Pin	Signal	Pin	Signal
1	VIN_19V	8	GPIO 42
2	+3.3 V	9	GPIO 41
3	+5 V	10	GPIO 40
4	VBUS	11	GPIO 39
5	RTC_INT (GPIO 14)	12	GPIO 38
6	I2C_SDA (GPIO 15)	13	GND
7	I2C_SCL (GPIO 16)	14	GND

J3 carries the power rails, the I²C bus with the RTC interrupt and five free GPIOs for extensions (for example a display or temperature sensors). The I²C lines are shared with the on-board RTC (address 0x68 is taken); the five GPIOs are not used by the current firmware. Its 3.3 V GPIOs are not 5 V tolerant.

J5 – LED chain extension (JST XH, 3-pin, 2.5 mm)

Pin	Signal	Description
1	+5 V	Supply for external WS2812B LEDs (from the +5 V rail, approx. 4.6–4.7 V)
2	DATA	Data output of LED2 through R16 (470 Ω)
3	GND	Ground

External LEDs draw their current from the +5 V rail. With USB-only supply, keep the total current within the 500 mA of the USB port (a WS2812B draws up to approx. 60 mA at full white).

JP1 – solder jumper "Custom_Settings" (GPIO 47)

Position	GPIO 47	Mode at the first start
Pads 1–2 bridged (as delivered)	high (3.3 V)	FactoryMode

Position	GPIO 47	Mode at the first start
Bridge 1–2 cut, pads 2–3 bridged	low (GND)	MakerMode
Bridge cut, all pads open	open	MakerMode

GPIO 47 is not a strapping pin, so every position is safe for booting. A change takes effect only after a device reset (see Device reset (section 5.5)).

10.4 Test points

TP1–TP11 are round SMD pads (1.5 mm diameter) for probes. The silkscreen shows the signal name next to each test point.

Test point	Signal	Expected value
TP3	19 V input	approx. 19 V (when supplied via J4)
TP4	5 V buck output (before diode)	5.0 V
TP1	+5 V rail	4.6 – 4.7 V
TP6	USB VBUS	5 V with USB connected
TP2	+3.3 V	3.3 V
TP5	GND	reference
TP7 / TP8	BOOT (GPIO 0) / RESET (EN)	3.3 V idle, 0 V when pressed
TP9 / TP10	RXD0 (GPIO 44) / TXD0 (GPIO 43)	UART0: boot messages of the ESP32 (3.3 V level, 115200 Bd)
TP11	VBAT_RTC (CR2032)	approx. 3.0 V with the cell inserted

10.5 GPIO assignment of the board

Function	GPIO	Function	GPIO
UART1 TX / RX (tester)	10 / 11	SD CLK / CMD	6 / 7
UART2 TX / RX (dongle)	8 / 9	SD D0 / D1 / D2 / D3	5 / 4 / 18 / 17
I²C SDA / SCL (RTC)	15 / 16	SD card detect (low = card)	13
RTC INT/SQW	14	LED1/LED2 data (WS2812B)	48
JP1 mode selection	47	Strapping pins, keep free	3, 45, 46
USB D− / D+	19 / 20	Expansion J3	38 – 42
BOOT	0	PSRAM (do not use)	35 – 37
UART0 RXD0 / TXD0	44 / 43	Not connected	1, 2, 12, 21

10.6 Pins and wiring rules

GPIO	Use	Accepted in Maker Settings
0, 3, 45, 46	Strapping pins (boot mode, JTAG, flash voltage)	accepted, but avoid
19, 20	Native USB (flashing, console)	rejected
26 – 32	Internal SPI flash	rejected
33 – 37	Octal PSRAM (N16R8); 33/34 also flash on octal-flash modules	rejected
43, 44	UART0 console (boot messages)	rejected

GPIO	Use	Accepted in Maker Settings
47	Mode selection; read only at the first start	accepted (pin is released after detection)
48	RGB LED on DevKit boards and the original board	accepted

- Use a level shifter between the 5 V tester port and the ESP32. A plain resistor divider on the RX line works, but the TX line needs a real driver for reliable 5 V levels.
- SD lines need 10 kΩ pull-ups to 3.3 V; keep the wires short (a few cm) or lower the SD frequency.
- The card-detect input must be high when a card is inserted (firmware 2.0.0). Without a switch, or with an active-low switch, enter −1.
- A DS3231 needs pull-ups of 2.2–10 kΩ to 3.3 V on SDA and SCL (most modules have them). Supply the module with 3.3 V so that its pull-ups do not pull the ESP32 pins to 5 V.
- For a WS2812 supplied with 5 V, make sure that 3.3 V data is recognised as high (supply the first LED via a diode as on the original board, or use a level shifter).

10.7 Bill of materials

26 line items, 57 components (53 SMD, 4 THT). Ready-to-upload BOM files for JLCPCB and PCBWay are in the repository (`docs/EBCTester_BOM_JLCPCB.csv` , `docs/EBCTester_BOM_PCBWay.xlsx`); check stock before ordering.

Ref.	Qty	Value / function	Package	Manufacturer / MPN	LCSC
U1	1	RTC with TCXO, I²C	SOIC-16W	Analog Devices DS3231SN#T&R	C9866
U2	1	Buck 19 V → 5 V, 2 A	TSOT-23-6	Diodes AP63205WU-7	C2071056
U3	1	Buck 5 V → 3.3 V, 2 A	TSOT-23-6	Diodes AP63203WU-7	C780769
U4	1	ESP32-S3 module, 16 MB/8 MB, u.FL	Module	Espressif ESP32-S3-WROOM-1U-N16R8	C3013946
U5	1	Level shifter 3.3 V / 5 V	TSSOP-14	TI TXS0104EPWR	C44955
LED1, LED2	2	RGB LED with driver	5050	Worldsemi WS2812B-B/W	C114586
J1	1	microSD socket, card detect	SMD	Hirose DM3AT-SF-PEJM5	C114218
J2	1	USB-C receptacle (USB 2.0)	SMD	HRO TYPE-C-31-M-12	C165948
J3	1	Header 14-pin, 2.5 mm	THT	BOOMELE XH-14A	C20200
J4	1	Header 6-pin, 2.5 mm	THT	BOOMELE XH-6A	C5663
J5	1	Header 3-pin, 2.5 mm (LED chain)	THT	BOOMELE XH-3A	C2316
D1, D2, D5	3	Schottky 40 V / 3 A	SMA	MDD SS34	C8678
D3	1	TVS 20 V, 600 W	SMB	TWGMC SMBJ20A	C726828
D4	1	USB ESD protection	SOT-23-6	ST USBLC6-2SC6	C7519
D6	1	LED yellow-green, power OK	0603	XINGLIGHT XL-1608SYGC-04	C965806
L1, L2	2	4.7 µH power inductor	6 × 6 mm	Bourns SRN6045TA-4R7M	C2044594
SW1, SW2	2	Tactile switch BOOT / RESET	5.1 × 5.1 mm	XKB TS-1187A-B-A-B	C318884
BT1	1	CR2032 holder	THT	MYOUNG BS-07-A1BJ001	C2979167
C1–C5, C12–C15, C17, C18	11	100 nF 16 V X7R	0402	TDK C1005X7R1C104KT000F	C338032
C6, C9	2	10 µF 50 V X7S	1206	Samsung CL31Y106KBKVPNE	C2803389

Ref.	Qty	Value / function	Package	Manufacturer / MPN	LCSC
C7, C8, C10, C11	4	22 µF 10 V X5R	0805	Samsung CL21A226KPCLRNC	C318688
C16	1	1 µF 10 V X7R	0402	Murata GRM155Z71A105KE01D	C528974
R1–R6, R9, R12, R13	9	10 kΩ 1 %	0402	Vishay CRCW040210K0FKED	C71617
R7, R8	2	4.7 kΩ 1 %	0402	Vishay RCC04024K70FKED	C1858949
R10, R11	2	5.1 kΩ 1 %	0402	Vishay CRCW04025K10FKTD	C4333224
R14, R15, R16	3	470 Ω 5 %	0402	Vishay CRCW0402470RJNED	C482183

Not assembled: antenna cable u.FL to SMA female bulkhead (kinghelm KH-IPEX-SMA-W-150, LCSC C2683343), 2.4 GHz antenna with SMA male (DreamLNK W10-2.4G, C381198), CR2032 cell, mating cables for J4 and J3 (XH housings with XH-T crimp contacts, UL1007 24 AWG). Use standard SMA, not RP-SMA. Insert the CR2032 with "+" facing up.

10.8 Printed circuit board

The following images were generated by the PCB manufacturer from the production data of the first batch (date code 20261103). They show the assembled board as ordered, with all components on the top side.

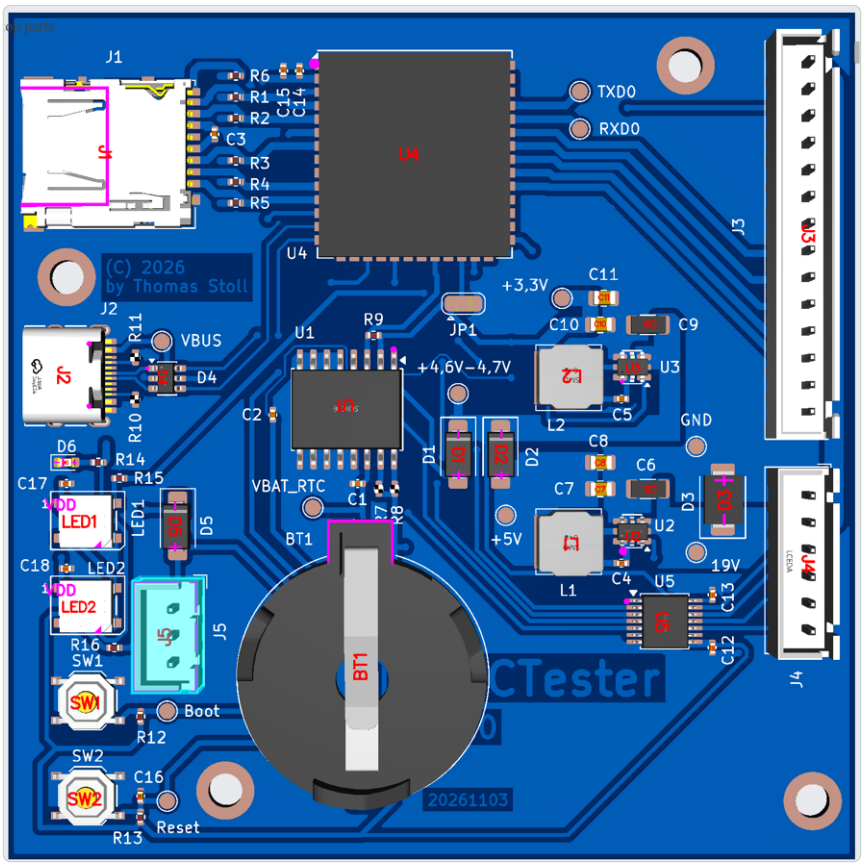


Figure 29: Assembled board, top view (manufacturer rendering)

Area	Components
Top left	J1 microSD socket with pull-ups R1–R6 and C3; test points TXD0/RXD0 (TP10/TP9) to the right of U4
Top centre	U4 ESP32-S3-WROOM-1U with its u.FL antenna socket; JP1 below it
Left edge	J2 USB-C with D4, R10/R11; below: D6 power LED, LED1/LED2 with C17/C18, J5 LED extension, SW1 BOOT and SW2 RESET

Area	Components
Centre	U1 DS3231 with R7–R9, C1/C2; BT1 CR2032 holder below
Right of centre	Power supply: U2 with L1, C4, C6–C8, D3 (19 V → 5 V); D1/D2 diode OR; U3 with L2, C5, C9–C11 (5 V → 3.3 V); U5 TXS0104E
Right edge	J3 expansion connector (top) and J4 tester connector (bottom)

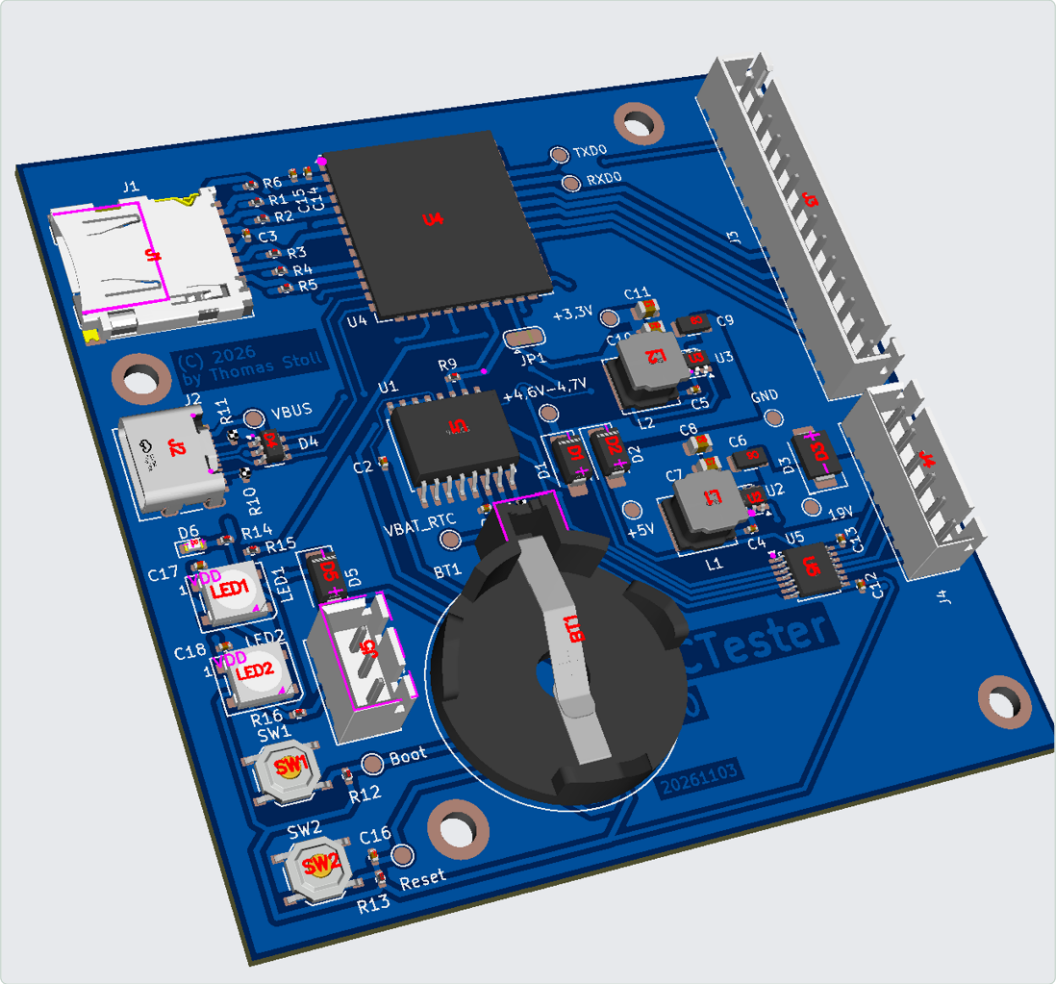


Figure 30: Oblique view from the bottom left: connectors J3/J4 on the right, coin-cell holder in front

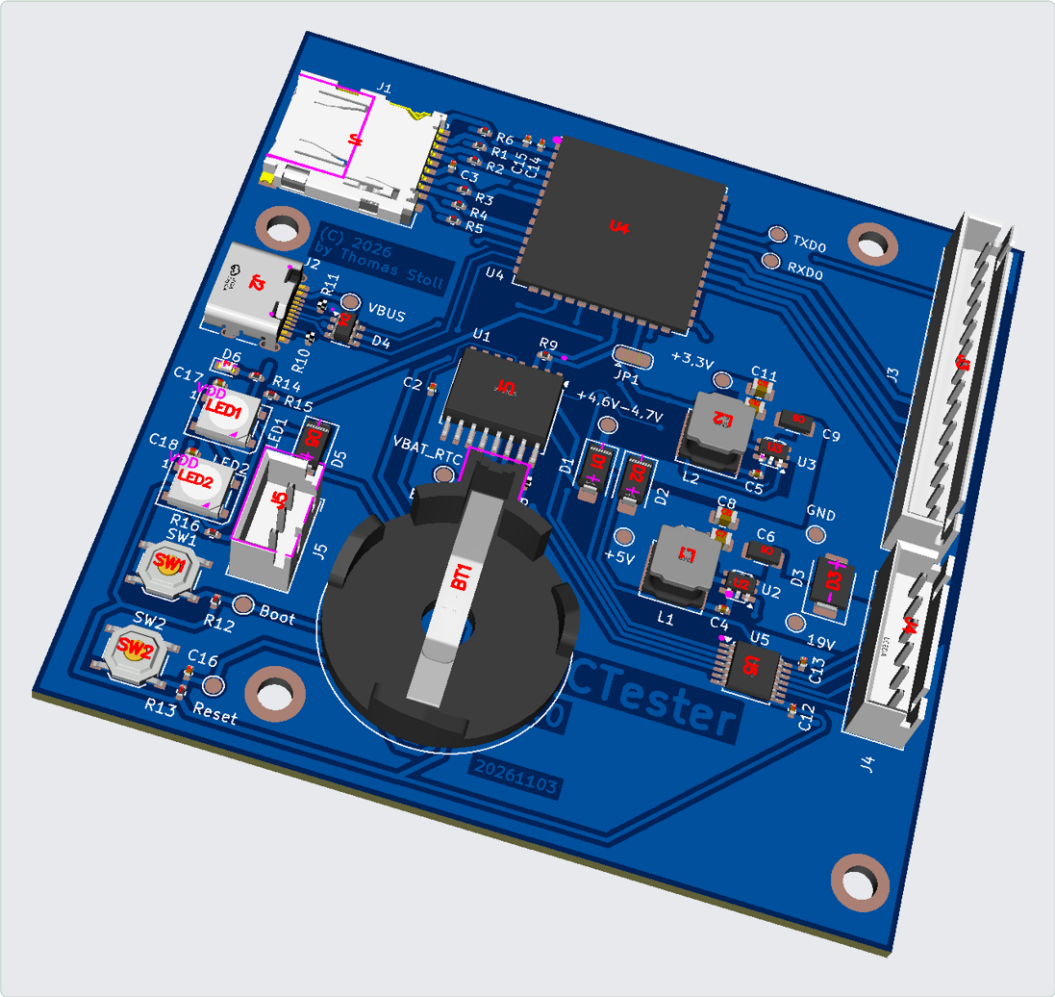


Figure 31: Oblique view: USB-C socket, status LEDs, J5 and the buttons on the left edge

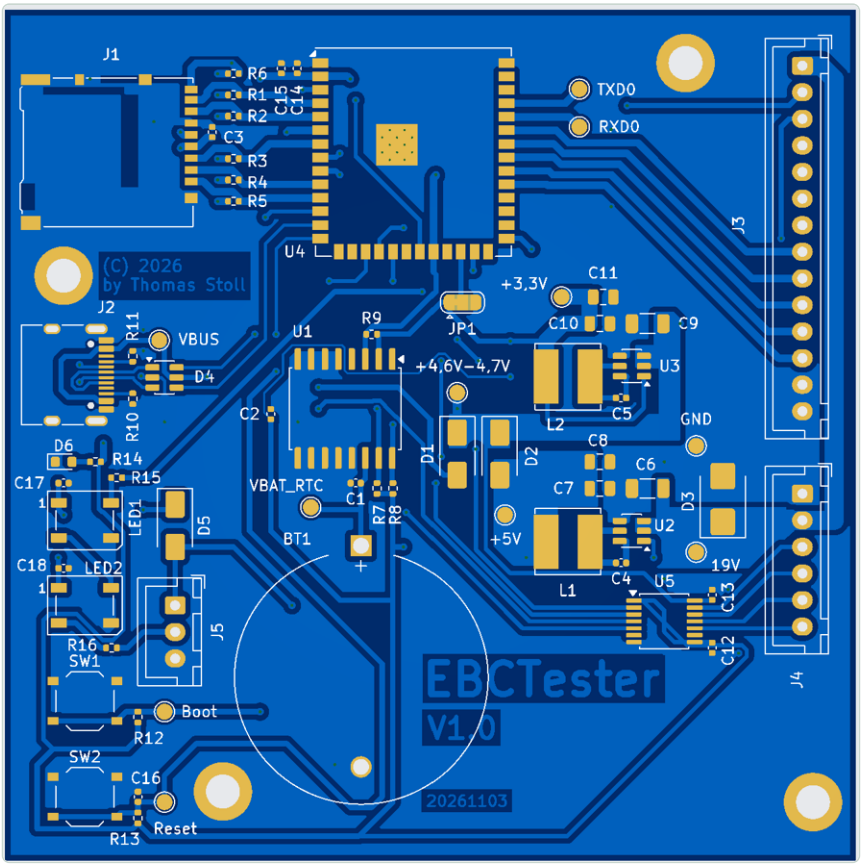


Figure 32: Layout, top side (copper and silkscreen)

Size	80 × 80 mm
Layers	2 copper layers, board thickness 1.6 mm, 35 µm copper
Assembly	All components on the top side: 53 SMD parts and 4 through-hole parts (J3, J4, J5, BT1)
Copper areas	Continuous GND plane on the bottom side; areas for +19 V, the 5 V buck output, +5 V and GND on the top side
Mounting holes	H1–H4, 3.2 mm for M3 screws, plated and connected to GND

Hole	x (mm)	y (mm)	Position
H1	64.0	5.0	top right, left of J3
H2	76.0	74.5	bottom right corner
H3	5.5	25.0	left edge, below the microSD socket
H4	20.5	73.5	bottom, right of the RESET button

Positions of the hole centres are measured from the top left corner of the board (orientation as in the layout figure, J3/J4 on the right). Because the mounting holes are connected to GND, a metal housing is connected to the ground of the board. The schematic is printed in Appendix C and published as `docs/EBCTester_Schematic.pdf`.

11 Building the hardware

This chapter shows how to build an EBCTester yourself. There are two ways:

Way	Suited for	Firmware mode
Original PCB	A compact, robust device with power supply from the EBC-A20, RTC and LEDs on board. Order the board assembled with the BOM files from the repository and the production data described in Hardware (chapter 10).	FactoryMode if JP1 is left bridged (1–2): no configuration needed
Module build	Quick start with ready-made breakout modules on a perfboard or breadboard. Every function of the original circuit can be replaced by a module.	MakerMode (GPIO 47 open): pins entered in Maker Settings

The circuit of the original board is the reference for both ways. The complete schematic is printed in Appendix C; its function is explained in Circuit description (section 10.2).

11.1 Example: the prototype

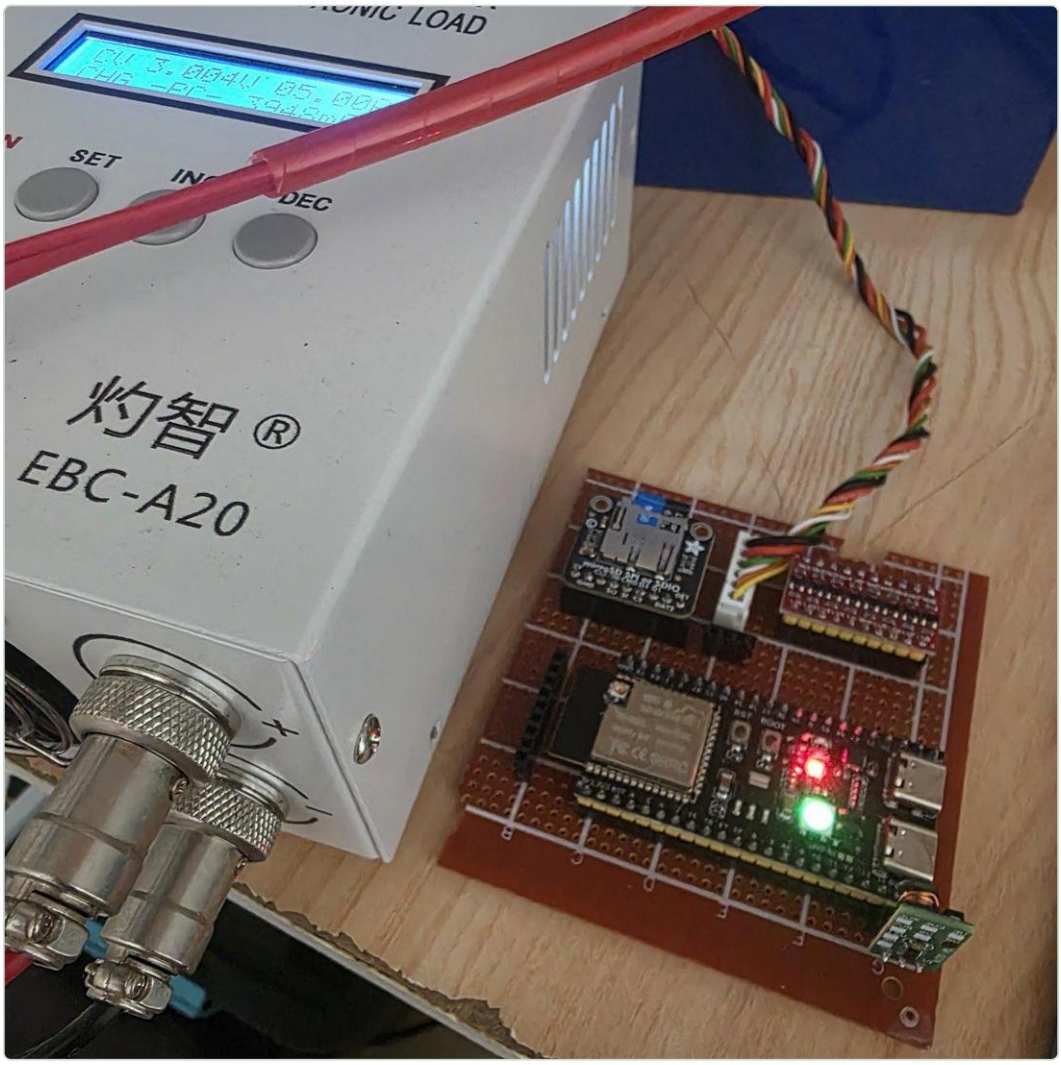


Figure 33: The author's prototype on a perfboard, connected to an EBC-A20 (CV mode, 3.004 V)

The first EBCTester was built from modules on a perfboard, and the firmware was developed on it. It is a good template for your own module build:

- **ESP32-S3 development board** (bottom) with 16 MB flash, two USB-C sockets, BOOT and RESET buttons and an RGB status LED. The LED on the development board is used as status LED.
- **microSD breakout** (top left) that supports both SPI and SDIO mode and has the pull-ups on board. The prototype uses it in SPI mode.
- **Bidirectional level shifter** (red module, right) between the 3.3 V ESP32 pins and the 5 V serial port of the tester.
- **Tester connector** (white JST socket) with a twisted cable to the serial port of the EBC-A20.

Pin assignment of the prototype (as entered in Maker Settings):

Function	GPIO	Function	GPIO
Tester UART TX / RX	10 / 11	SD bus	SPI, 10 MHz
Dongle UART TX / RX	8 / 9	SPI CLK / MOSI	6 / 7
Status LED (on the development board)	48	SPI MISO / CS	5 / 17

These are the same pins as on the original board: in SPI mode, MISO uses the D0 line, MOSI the CMD line and CS the D3 line of the SDMMC wiring. A board wired like this can therefore be switched between SPI and 4-bit SDMMC in Maker Settings if all six SD lines are connected.

11.2 Replacing board components by modules

Original board	Module replacement	Notes
U4 ESP32-S3-WROOM-1U-N16R8, J2 USB-C, SW1/SW2, D4	ESP32-S3 development board with N16R8 module (for example ESP32-S3-DevKitC-1 N16R8 or compatible)	16 MB flash is mandatory. Boards with PCB antenna need no external antenna. Check which GPIO the on-board RGB LED uses (often 48, on some revisions 38).
U2, U3, D1, D2, D3 power supply	USB supply of the development board, or a 19 V → 5 V step-down module feeding its 5 V pin	Do not feed 5 V into the 5 V pin while USB is connected unless the board has a protection diode. The 3.3 V regulator of the development board then supplies the logic.
U5 TXS0104E	Bidirectional level-shifter module (TXS0104E/TXS0108E or MOSFET type)	Low side (VA/LV) to 3.3 V, high side (VB/HV) to 5 V, OE to 3.3 V (enabled). Only the tester UART (2 channels) is needed; UART2 is reserved.
J1 microSD, R1–R6	microSD breakout for 3.3 V with pull-ups	Breakouts that support SDIO allow 4-bit SDMMC. Simple Arduino modules with a 5 V regulator and level buffer support SPI only.
U1 DS3231, BT1, R7–R9	DS3231 module (optional)	Supply with 3.3 V. Many modules contain a charging circuit for a rechargeable LIR2032; with a non-rechargeable CR2032 remove the charging diode or resistor as described for the module.
LED1, LED2, J5	RGB LED of the development board, or a WS2812 strip	Up to 16 LEDs in one chain. A 5 V WS2812 needs a valid high level: feed the first LED through a diode or use a level shifter.
JP1	–	Leave GPIO 47 open (or connect it to GND) so that the device starts in MakerMode.
J4	Any 4-pin connector or soldered cable	GND, tester RX, tester TX are required; 19 V is only needed if the board is powered from the tester.

11.3 Wiring

Tester interface

ESP32-S3	Level shifter	Tester (5 V TTL)
3.3 V	VA / LV, OE	–
5 V	VB / HV	–
GND	GND	GND
Tester UART TX (GPIO 10)	A1 → B1	RX of the tester
Tester UART RX (GPIO 11)	A2 ← B2	TX of the tester

> **Caution**
> Never connect a 5 V tester line directly to an ESP32 pin. The pinout of the tester port differs between models and cable versions – measure it with a multimeter before connecting (idle level of the tester's TX line: about 5 V).

microSD card

Card signal	4-bit SDMMC	SPI	Default GPIO
CLK	CLK	CLK (SCK)	6
CMD	CMD	MOSI (DI)	7
DAT0	D0	MISO (DO)	5
DAT1	D1	–	4
DAT2	D2	–	18
DAT3 / CD	D3	CS	17
VDD / GND	3.3 V / GND, 100 nF close to the socket; 10 kΩ pull-ups to 3.3 V on CMD and DAT0–DAT3		

Real-time clock and LEDs

Signal	Default GPIO	Notes
DS3231 SDA / SCL	15 / 16	Pull-ups to 3.3 V; module VCC to 3.3 V
DS3231 INT/SQW	14	Optional, reserved; `-1` if not connected
WS2812 DIN	48	Series resistor 330–470 Ω close to the first LED

The pins in these tables are the firmware defaults and match the original board – using them means you only have to activate the configuration in Maker Settings. Any other free, output-capable GPIO can be used (see Pins and wiring rules (section 10.6)).

11.4 Commissioning

1. **Check the wiring** without power: no short circuit between 5 V, 3.3 V and GND; no 5 V line on an ESP32 pin.
2. **Power up without SD card and tester.** Check 3.3 V and 5 V (on the original board at TP2 and TP1, see Test points (section 10.4)).
3. **Install the firmware** over USB (Installing the firmware (section 2.2)). The device starts in MakerMode with an inactive configuration.

4. **Enter the pins** in Maker Settings, first *without* ConfigAktive, and save. Then tick ConfigAktive and save again.
5. **SD card:** insert a FAT32 card and open **Administration → SD card**. "SD card ready" and the free space must be shown; `ebc_a20.db` is created.
6. **Tester:** select the model in **Settings → EBC tester**, switch the tester on and click **Connect** in the Manual tab. The RX indicator turns green and the message counter increases.
7. **Real-time clock** (if fitted): enable it in Maker Settings, connect the device to Wi-Fi with Internet access and check the debug log for RTC messages and the clock in the header. Then disconnect the power for a minute – after the restart the time must still be correct.
8. **LEDs:** set the LED count in **Settings → LEDs**; the tester-mode LED must turn green when the tester is connected.
9. **First test** with a low current (for example a short D-CC discharge at 0.5 A) and check the recorded run in Measurements.

12 Firmware installation and build

12.1 Installing a release

Each GitHub release contains `EBCTester-<version>-full-install.bin` with bootloader, partition table and application. Flash it at address `0x0` with esptool (see Installing the firmware (section 2.2)). Updates are normally installed over Wi-Fi; a USB installation is only needed for the first installation, after a change of the partition table or for recovery.

12.2 Building from source

Requirements: VS Code with the ESP-IDF extension, ESP-IDF 6.0.1, Git and an Internet connection for the first build (managed components and the SQLite source are downloaded and hash-checked).

```
git clone --recurse-submodules https://github.com/Gemelon/EBCTester.git
# existing checkout:
git submodule update --init --recursive
```

1. Open the project in VS Code and select the ESP-IDF environment and target `esp32s3`.
2. Review the defaults in menuconfig (see below).
3. Build with **ESP-IDF: Build your project**.
4. For the first installation, flash bootloader, partition table and application over USB.

A ZIP download of the repository does not contain the submodules. Do not run `git submodule update --remote` without reviewing the changes – the main repository records the tested versions. `sdkconfig.defaults` provides portable defaults; an existing local `sdkconfig` takes priority. Size optimisation (`-Os`) is the default.

12.3 Build configuration (menuconfig)

Since firmware 1.99.3 the following options only define the **defaults of a new configuration**. After the first start, the firmware uses the values stored in NVS. Changing an option therefore has no effect on a device that already has a valid configuration – reset the device configuration to apply new defaults.

Menu	Option (sdkconfig)	Default
EBCTester → EBC battery tester UARTs	Tester / dongle UART TX, RX, baud (`CONFIG_EBC_TESTER_UART_*` , `CONFIG_EBC_DONGLE_UART_*`)	see Firmware defaults (section 9.5)
EBCTester → Battery test control	Averaging length (`CONFIG_TEST_CONTROL_AVERAGING_LENGTH`)	20
EBCTester → Battery test store	Device ID; max. pending samples (build option only)	EBC-A20; 3000
EBCTester → Home Assistant MQTT	Initial MQTT state (`CONFIG_EBCTESTER_HA_MQTT_ENABLE`)	off
SD Card Manager	Bus mode, frequencies, pins, card detect, mount point	see Firmware defaults (section 9.5)
OTA Suite network	AP SSID prefix (build option only), AP password (empty = random), SNTP server	BatteryTester, ebctester, pool.ntp.org
OTA Suite web interface	Initial web password, settings confirmation, SD languages, primary tabs, tab order	ebctester, on, on
OTA Suite update checks	Firmware manifest URL	release channel

Menu	Option (sdkconfig)	Default
OTA Suite firmware updates	Allow unencrypted HTTP OTA	off
OTA Suite status LED	Enable WS2812 LED, GPIO	on, 48
EBCTester → DS3231 real-time clock	INT, SDA, SCL GPIO (`CONFIG_EBCTESTER_RTC_INT_GPIO` , `…_I2C_SDA_GPIO` , `…_I2C_SCL_GPIO`)	14, 15, 16

12.4 Partition table

Name	Type	Offset	Size	Use
nvs	data/nvs	0x9000	24 KB	Settings (device configuration, Wi-Fi, model, plan, battery …)
otadata	data/ota	0xF000	8 KB	Active firmware slot
phy_init	data/phy	0x11000	4 KB	RF calibration
ota_0 / ota_1	app	0x20000 / 0x590000	2 × 5.4 MB	Two firmware slots for safe updates
storage	data	0xB00000	5 MB	Reserved, currently unused

A changed partition table cannot be installed over the air; it requires flashing over USB. Back up settings and data before repartitioning.

12.5 Status LEDs

The firmware drives a chain of 1–16 WS2812 RGB LEDs (GRB) on the configured GPIO; on the original board LED1 and LED2, extendable through J5. The LED task updates every 50 ms; brightness is kept low (normal colours at about 24/255). The function group of each LED is stored in the NVS namespace `leds` .

Function group	Input	Colours
Tester mode	Connection, manual operation, automatic plan	off / green / blue / orange
Message frame status	Counter of valid frames, age of the last valid and invalid frame, model check	off after 5 s silence; dim green; one 100 ms bright-green flash per valid frame (100 ms pause between flashes); orange on invalid frames or model errors
Off	–	off

Update state (LEDs with OTA override)	LED
Firmware download	Blue pulse (2 seconds)
Verification	Yellow pulse (may be brief)
Waiting for restart	Solid green
Update failed	Red blink (0.5 s on/off) until a new attempt or restart
Configuration inactive or LED disabled	All LEDs off

13 Communication with the tester

13.1 Serial interface

UART1 is connected to the tester with 9600 baud, 8 data bits, odd parity, 1 stop bit (8O1) at 5 V TTL level. Frames start with `0xFA` and end with `0xF8`. Status frames are 19 bytes long. An XOR checksum over bytes 1–16 is verified before any data is used. The EBC-A20 transmits checksums in the range 0xF0–0xFF with the upper nibble cleared, to avoid the frame markers; this variant is also accepted.

Set-points are transmitted as two bytes in base 240 (value = high × 240 + low) with a resolution of 0.01 A or 0.01 V. The range is sufficient for 50 A; the 10 mA resolution has not been verified on real A50L hardware yet. The discharge command also carries a time limit in whole minutes.

13.2 Model detection

Byte 16 of each status frame contains the model identifier (normalised as raw value modulo 100). For status bytes of at least 100, bytes 10 and 11 contain the tester's firmware version (B10 × 240 + B11), which distinguishes variants with the same identifier.

Ident (dec / hex)	Firmware version	Model	Enabled
5 / 0x05	302 / other	EBC-A05+ / EBC-A05	no
6 / 0x06	310 / other	EBC-A10+ / EBC-A10H	no
9 / 0x09	311 / other	EBC-A20+ / EBC-A20	A20 only
10 / 0x0A	311 / other	EBC-A50L / EBC-A40L	yes
20 / 0x14	310 / other	EBD-M10 / EBD-A10H	no
25 / 0x19	310 / 315 / other	EBD-M03+ / EBD-M05 / EBD-M03	no
38 / 0x26	310 / 311 / other	EBD-A100L / EBC-A100L / EBD-A100	no

While the firmware version of an ident-10 tester is not yet known, it is shown as "EBC-A40L/A50L (Version unknown)" and the conservative A40L limits are used. The model check runs on every valid frame. On a mismatch with the selected model, the connection and measurement state are cleared, pending commands are discarded, Disconnect is sent and automatic reconnection is suppressed.

13.3 Operating modes

Code	Mode	Code	Mode	Code	Mode
0x00	D-CC stopped	0x0A	D-CC active	0x14	D-CC completed
0x01	D-CP stopped	0x0B	D-CP active	0x15	D-CP completed
0x02	C-CV stopped	0x0C	C-CV active	0x16	C-CV completed

14 Data storage

14.1 SQLite database on the SD card

The database file is `/sdcard/ebc_a20.db`. It is created when a card is mounted; if the device starts without a card, it is created as soon as one is inserted. The card is never formatted and an existing database is never deleted. A file without a valid SQLite header is preserved under the name `ebc_a20.db.invalid-XXXXXXXX` before a new database is created.

Table	Content
ebc_battery	Batteries (label, manufacturer, serial number, date, nominal capacity, notes)
ebc_test_plan / ebc_test_plan_step	Saved test plans and their steps
ebc_test_run	Test runs (battery, plan, start/end, status, current step, cycle, firmware, notes, display name)
ebc_run_segment	Segments of a run (mode, step, cycle, set-points)
ebc_measurement	Measurements (time UTC and uptime, current, voltage, capacity …)
ebc_resistance_result	Three DCIR results per test (times, U1/I1, U2/I2, mΩ, validity)
ebc_schema_metadata, ebc_sync_state	Schema version; MariaDB synchronisation progress

The views `ebc_measurement_view` and `ebc_resistance_result_view` make evaluation easier. SQLite runs with `synchronous=FULL`, real `fsync()`, foreign keys and an integrity check at start-up.

Schema	Firmware	Change
1	–	Base tables (corresponds to MariaDB migrations 001–005)
2	DCIR support	Table ebc_resistance_result
3	1.4.67 and later	Column display_name for custom run names

Migrations run automatically in a transaction at the first start with a card. Older firmware refuses a newer schema: keep a database backup before an update if you might need to downgrade.

Time stamps are stored in UTC and require a synchronised clock (SNTP or an enabled, valid DS3231). Without it, the device uptime is recorded instead; no calendar time is invented.

14.2 MariaDB backup database

The MariaDB synchroniser copies plans, steps, batteries, runs, segments, measurements and DCIR results to an existing MariaDB schema (for example `ebc_a20`). Progress is tracked per run; only new data is sent in batches of 200 rows with `INSERT IGNORE`.

- The base schema (migrations 001–005) must exist on the server. The repository contains migration `sql/006_resistance_results_mariadb.sql` for DCIR results, not a complete initial schema.
- The user needs a `mysql_native_password` account with SELECT/INSERT/UPDATE on `ebc_test_plan`, `ebc_battery`, `ebc_test_run`, additionally DELETE on `ebc_test_plan_step`, and INSERT on `ebc_run_segment` and `ebc_measurement`, plus rights on the DCIR result table.
- The connection is plain TCP without TLS.

```
SELECT battery_label, test_started_at_utc, requested_seconds,
       actual_seconds, resistance_mohm, valid, error
FROM ebc_resistance_result_view
ORDER BY test_started_at_utc DESC, run_id, result_number;
```

15 Interfaces

15.1 HTTP API

The web interface uses a JSON API that can also be used by your own scripts. With browser login enabled, all endpoints require a login session (cookie); POST requests additionally require the CSRF token from `GET /api/status` in the header `X-CSRF-Token`. Saving settings also requires `X-Admin-Password` when the password confirmation is enabled.

Endpoint	Method	Purpose
/api/auth/login, /api/auth/logout	POST	Login (JSON user name/password, header `X-Web-Login: 1`) and logout
/api/status	GET	Device, network and update status, CSRF token, `configActive`, `firmwareMode` (`factory` / `maker`), tabs, `clockEnabled` and `clockUtc` (UTC seconds, 0 = no valid time)
/api/tester/status	GET	Live values, connection, model and limits, plan, recorder
/api/tester/command	POST	connect, disconnect, stop, auto_connect, charge, discharge_cc, discharge_cp
/api/tester/plan, /api/tester/plan/action	POST	Save plan; start, abort, reset, load_db, save_db
/api/db/runs, /api/db/run?id=, /api/db/measurements?id=&max=	GET	Run catalogue, run details and segments, measurement series
/api/auth/settings	POST	Login settings: `sessionDays` (1/7), `loginEnabled`, optional new password
/api/tester/settings, /api/tester/catalog	GET	Manual values, battery, plan; saved batteries and plans
/api/tester/battery, /api/tester/model	POST	Battery actions; tester model selection
/api/tester/resistance(/history)	GET/POST	DCIR configuration, status, start/abort; history
/api/test-runs(/rename|/delete)	GET/POST	Test-run management
/api/maker/config	GET/POST	Maker Settings (MakerMode only, otherwise HTTP 403). GET returns the field list with ranges; passwords only as `configured`. POST: `{"active":bool,"tester":{…},"ota":{…},"sd":{…},"rtc":{…}}`, restarts the device
/api/leds/settings	GET/POST	LED count (1–16) and per-LED `group` (`off`, `tester`, `frames`) with `ota` override; applied immediately
/api/config/reset	POST	Device reset: `{"scope":"config"|"all","confirm":true}`, restarts the device; HTTP 409 while a test or update is running
/api/mariadb/…, /api/mqtt/…, /api/debug/…	GET/POST	Status and settings of the optional services
/api/sd/…, /api/updates/check, /api/ota	GET/POST	SD card file operations; update check and installation

`GET /api/db/measurements` returns points `[segment, uptimeMs, epochMs, I, U, Ah]`, reduced to `max` points (200–6000).

15.2 Home Assistant MQTT topics

On connection the device publishes a retained discovery document to `homeassistant/device/ebctester_<MAC>/config`, its availability (`online` / `offline` as Last Will) to `<prefix>/availability`, and every recorded measurement (voltage, current, capacity) as a non-retained QoS 0 message to `<prefix>/state`. Mode, plan status and battery entities are already announced but do not yet receive values. Commands are accepted on `<prefix>/command` as a JSON object:

```
{"command":"start_plan"}        (only if remote start is allowed)
{"command":"stop"}
{"command":"abort"}
{"command":"select_plan","name":"Plan name"}
{"command":"select_battery","name":"Battery label"}
```

MQTT cannot set free set-points or create database entries. Invalid commands are ignored. Failed connections are retried with increasing intervals from 1 to 60 seconds.

16 Firmware updates and publishing

16.1 Update procedure

- The device downloads only the manifest `firmware.json` (after Wi-Fi connects, every six hours, when the UI is opened). HTTPS checks wait for a synchronised clock. Inactive Maker devices do not check.
- Versions must be strictly numeric `major.minor.patch` and newer than the installed one.
- Manifest and image must be delivered directly (HTTP 200, no redirects). HTTPS validates certificate chain, host name and dates. HTTP is only possible with the explicit option for a trusted test network.
- The image is streamed into the inactive slot; exact size and SHA-256 are checked before it is selected for booting. Project name and target must match.
- After restart the new firmware must run for 10 seconds; an earlier reset or crash rolls back to the previous version.

16.2 Manifest format

```
{
  "project": "EBCTester",
  "target": "esp32s3",
  "version": "2.0.1",
  "url": "https://updates.example.org/BatteryTester/EBCTester-2.0.1.bin",
  "size": 2825440,
  "sha256": "<64 lower-case hex digits>",
  "notes": "Release notes"
}
```

All seven fields are mandatory; the manifest must be smaller than 2048 bytes, notes at most 512 bytes. The image URL must be in the same directory as the manifest and named `<project>-<version>.bin`.

16.3 Publishing your own builds

Publishing is disabled in a fresh checkout. It is enabled with a local, Git-ignored file `CMakeUserConfig.cmake` (see README). A successful build then creates the immutable versioned image, a compatibility copy and finally the manifest, optionally uploaded by SFTP. A published version is never overwritten with different content – increase the version first. Always distribute the license directory together with the firmware.

17 Security and limitations

17.1 Security

- **Login:** user `admin` with the login password (initially `ebctester`). The password, the session duration and the login switch are stored in the device configuration (`ebc_cfg`). Sessions use a random 256-bit cookie (`HttpOnly`, `SameSite=Lax`), last exactly 1 or 7 days and are held in RAM (a restart ends them). At most 8 sessions at a time; failed logins are delayed by one second.
- **Settings protection:** all changing requests require a CSRF token; settings additionally require the current login password, independent of whether browser login is enabled. Password or login-policy changes invalidate all sessions.
- **Access point:** set a personal access-point password in Maker Settings; the firmware default is the same on every device.
- **Transport:** the web interface, MQTT and MariaDB are unencrypted. Use only trusted networks, never forward port 80 to the Internet.
- **Storage:** NVS and flash are not encrypted by default; protect physical access and the USB port (the start-up log shows a generated access-point password). Change the shared initial login password `ebctester` on first use.
- **Firmware:** SHA-256 protects against corruption, not against deliberate replacement. Secure Boot, signed images and flash encryption are separate deployment decisions.

17.2 Known limitations (firmware 2.0.0)

- The UART2 bridge for the manufacturer's PC software is not implemented.
- The real-time clock is implemented; long-term tests of the backup cell on the original board are still pending.
- The INT output of the DS3231 is reserved; alarms and deep-sleep wake-up are not implemented.
- Only EBC-A20, EBC-A40L and EBC-A50L can be controlled; physical tests with the A50L are pending.
- No battery-chemistry profiles and no temperature measurement.
- MQTT publishes measurements only; status entities do not yet receive values.
- The card-detect input is active-high only.
- A forgotten login password can only be reset by erasing the settings memory over USB.

17.3 License

Copyright 2026 Thomas Stoll. EBCTester, OTA Suite and SD Card Manager are licensed under the Apache License 2.0 – commercial use, modification and sale are permitted under its terms; its warranty and liability disclaimers apply. Third-party components keep their own licenses: Bootstrap 5.3.3 and Chart.js 4.4.9 (MIT), SQLite (public domain), ESP-IDF (Apache 2.0) and others, listed in `THIRD_PARTY_NOTICES.md`. EBCTester is an independent project and not an official ZKETECH product; product names are trademarks of their respective owners.

Appendix A – Language files

Additional user-interface languages are loaded from the folder `/languages` on the SD card. A complete template is `sd_card/languages/en-custom.json` in the repository; a partial Filipino (Tagalog) example is `fil.json`.

1. Copy the template, for example as `fr.json`.
2. Set `"code": "fr"` and `"name": "Français"`.
3. Translate only the values in `"strings"`; keep all keys unchanged.
4. Keep placeholders such as `{p0}`, `%u`, `%s` (printf placeholders in the same order).
5. Save as UTF-8 without BOM, upload to `/languages/fr.json` (for example in the SD card tab), click **Reload languages** and select the language in the header.

```
{
  "version": 1,
  "code": "fr",
  "name": "Français",
  "strings": {
    "language.title": "Langue",
    "...": "..."
  }
}
```

Missing keys fall back to English. Files named `en.json` or `de.json` replace the built-in texts while present. Limits: max. 16 files, 128 KB per file, 1024 entries, 4096 bytes per value, 80 bytes for the name. Language codes use lower-case letters, digits and hyphens (max. 24 characters), for example `fr`, `en-gb`. Loading languages from SD requires the option **Load additional languages from SD** in Maker Settings and an active configuration.

Appendix B – Glossary

AP	Access point – the device's own Wi-Fi network for setup and fallback access.
C-CV	Charging with constant current, then constant voltage (CC/CV).
ConfigAktive	Stored switch that enables the hardware configuration. When off, only the access point and the web interface run.
CSRF	Cross-site request forgery; the token protects against requests from foreign web pages.
D-CC / D-CP	Discharging with constant current / constant power.
DCIR	Direct-current internal resistance, measured with a load pulse.
FactoryMode	Firmware mode of the original board (GPIO 47 high): ready to use with the firmware defaults.
Frame	One message between EBCTester and the tester.
MakerMode	Firmware mode of self-built devices (GPIO 47 low or open): hardware configured in Maker Settings.
NVS	Non-volatile storage in the ESP32 flash for settings.
NTP	Network Time Protocol – see SNTP.
OTA	Over-the-air – firmware update over Wi-Fi.
RTC	Real-time clock – the DS3231 chip that keeps the time while the device is off.
Segment	Section of a test run with one operating mode.
SNTP	Simple Network Time Protocol – time synchronisation over the Internet.
SOC	State of charge in percent.
Test run	Recording of one manual operation, plan or DCIR test with all its measurements.
UTC	Coordinated Universal Time – the time base of all stored time stamps.
WS2812	RGB LED with integrated driver; several LEDs are connected in a chain on one data line.

Appendix C – Schematic

Schematic of the EBCTester board, revision 1 (2 November 2026). The page is rotated; the original A3 drawing is published as `docs/EBCTester_Schematic.pdf` in the repository and can be zoomed without loss.

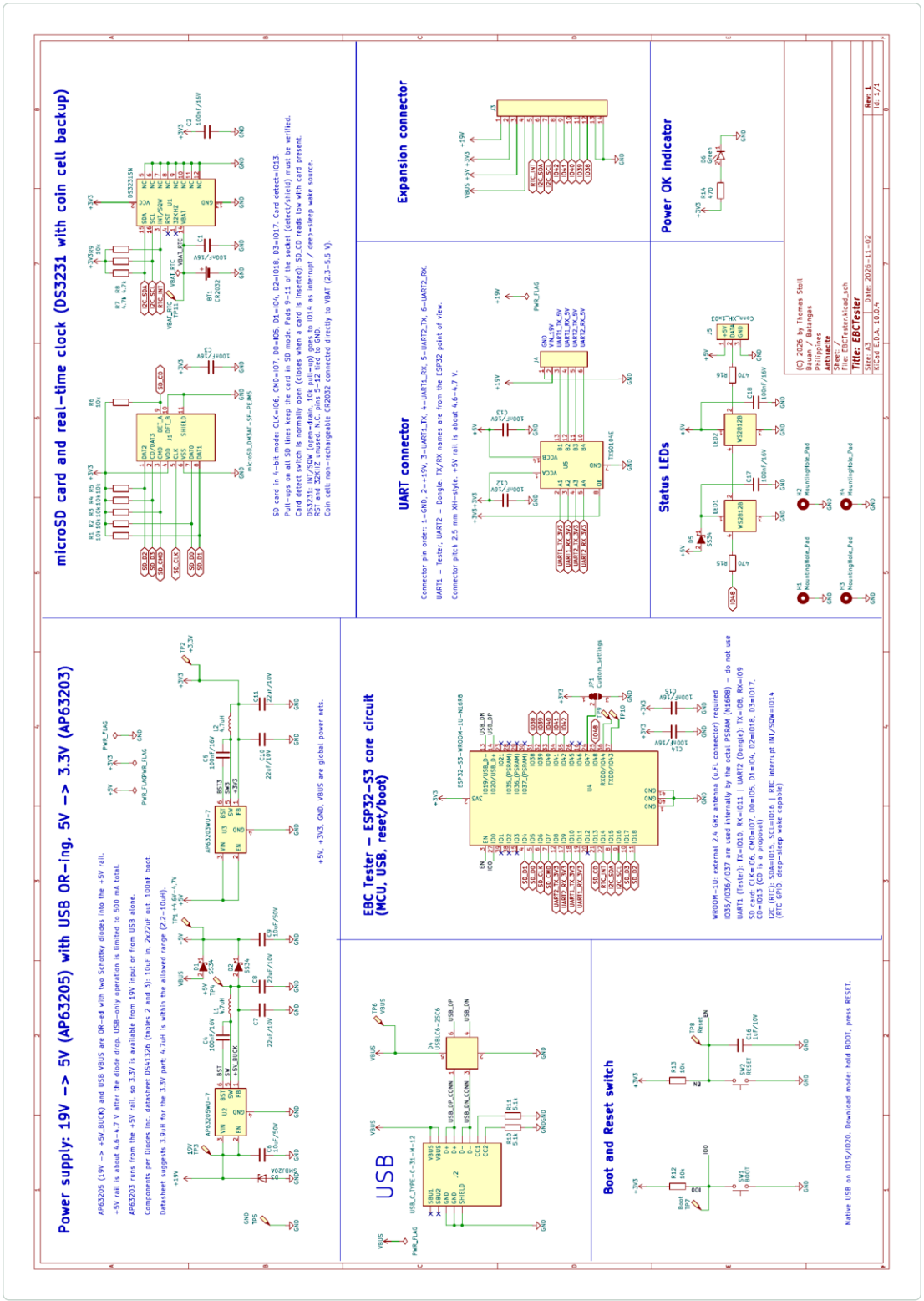


Figure 34: EBCTester schematic, revision 1 (KiCad 10)